



GitLab Communication

We're an all-remote company that allows people to work from almost anywhere in the world. It's important for us to practice clear communication in ways that help us stay connected and work more efficiently.

We're an [all-remote](#) company that allows people to work from [almost anywhere in the world](#). We hire great people regardless of where they live, but with GitLab team members across [more than 60 countries](#), it's important for us to practice clear communication in ways that help us stay connected and work more efficiently.

To accomplish this, we use [asynchronous communication](#) as a **starting point** and stay as open and transparent as we can by communicating through public issues, [merge requests](#), and [Slack channels](#).

We also place an emphasis on ensuring that conclusions of offline conversations are written down. When we go **back and forth three times**, we jump on a [synchronous video call](#).

We communicate respectfully and professionally at all times.

Effective & Responsible Communication Guidelines

1. **Assume [Positive Intent](#).** Always begin with a position of positivity and grace.
2. **Kindness Matters.** You are looking at a screen, but you are really talking to a person. If you wouldn't say it to a person's face, do not send it to them in a text message.
3. **Express Your Thoughts Responsibly and Inclusively.** We live in different locations and often have very different perspectives. We want to know your thoughts, opinions, and feelings on things. We also ask you to consider the guidelines around communicating potentially sensitive topics.
4. **Own It.** If you say it or type it, own it. If it hurts the company or an individual, even unintentionally, we encourage you to look at things from other points of view and apologize easily.

5. **Be a Role Model of our [Values](#).**
6. **Feedback is Essential.** It is difficult to know what is appropriate in every one of our team members 60+ countries. We encourage team members to give feedback and receive feedback in a considerate way.
7. **Do not underestimate a 1:1.** Asynchronous communication (e.g., via text) is helpful and necessary. In some cases (e.g., to clarify misunderstandings) it can be much more effective to jump on a Zoom video call.
8. **Always Adhere to our [Anti-Harassment Policy](#) and [GitLab Code of Business Conduct and Ethics](#).** Everyone should be comfortable in their work environment.
9. **Focus on what we can directly influence.** There are many factors we can't directly influence and we should avoid spending time discussing those things. For example, we don't talk about our [market capitalization](#) because aspects of this are out of our control. Instead, we should focus on how we can work together to achieve company objectives and grow [annual recurring revenue](#).
10. **Commit to [active and effective listening](#).**

Asynchronous communication

Asynchronous communication is the art of communicating and moving projects forward *without* the need for additional stakeholders to be available at the same time your communicate is sent.

In distributed systems, async and sync communication aren't ideologies. They're tools, each with a job. Async messaging (queues, pub/sub, event streams) is how you scale. It decouples producers from consumers, absorbs bursty load, and lets services keep working when peers are slow or down. Sync calls (RPC, request/response) are how you coordinate when correctness depends on a fresh, agreed-upon answer right now: a payment authorization, a lock acquisition, a leader election. Build everything async and you get a system that scales beautifully but can't make a decision. Build everything sync and you get a system that's correct but brittle, where one slow node stalls the whole graph.

The same pattern applies to how we work. Slack, docs, and GitLab issues are our async layer. They scale across timezones, create durable artifacts, and let people work without blocking on each other. That's most of the work, and it should be. But some decisions require a synchronous transaction. Aligning on a strategy shift, resolving a real disagreement, building trust with a new exec, debugging a problem where the context isn't yet legible enough to write down, these are the moments where the round-trip latency of async (a thread that takes four days and seventeen replies to converge) is worse than thirty minutes of sync. The failure mode of "async-only" is the same failure mode as an event-driven architecture with no synchronous endpoints. You get eventual consistency on decisions. Threads fork, context drifts, no one knows who

decided what, and the work that needed a clean commit point becomes a slow-moving consensus problem instead.

Team Member Resource: To learn more on when to use asynchronous and synchronous communication, review this full [overview of async vs. sync with additional guidance](#).

Everyone is a moderator

If you see something that concerns you in Slack, Issues, Merge Requests, Video, Emails or any other forum, we encourage you to respectfully say something directly to the individual in a 1:1 format.

If there is an issue to raise regarding someone's communication or conduct, team members should follow the process for [raising communication concerns](#) by sharing their concern with their manager or, if preferred, email Team Member Relations (teammemberrelations@gitlab.com) directly.

Communicating When Using Generative AI Tools

1. **[Own your output](#):** You are responsible for the accuracy, appropriateness, and value of any content you share, regardless of how it was created. As the ultimate author, you should read, understand, and refine comments, code or other artifacts before asking others to review.
 - **Verify before sharing:** AI can generate plausible but incorrect information. Always verify facts, code functionality, and technical details before posting.
 - **Use [inclusive language](#):** Review AI outputs for bias and ensure they align with our [Diversity, Inclusion & Belonging](#) value.
 - **[Keep broadcasts short](#):** AI tools can generate verbose and repetitive content. Edit for brevity and clarity, especially in one-to-many communications. Optimize for [efficiency for the right group](#). Consider your audience's time when deciding what to share.
2. **Protect confidential information:** Be aware of [Data Classification Standards](#) for the data and AI tooling you're using. Use only approved internal tools for sensitive content.
3. **Maintain authenticity:** Ensure your communications reflect genuine human judgment and align with our [values](#). This is especially important in customer interactions and when [providing feedback](#).
4. **Don't use AI disclosure to deflect responsibility:** Consider your intent before mentioning AI use. Transparency should add value, not serve as a disclaimer reducing your accountability. If you're mentioning it to hedge against errors, your content may not be ready to share.

AI tools can enhance [efficiency](#), but they don't reduce your responsibility to communicate effectively and professionally.

If you're a recipient of content you think falls short of this guidance (sometimes called "AI slop"), operate on our principles of ["assume positive intent"](#), ["negative feedback is 1-on-1"](#), and ["give feedback effectively"](#).

Communicating Potentially GitLab Sensitive Topics

(This guidance supplements and overlaps with [GitLab's SAFE Framework](#), the guidance on the use of [the internal handbook](#), and the additional guidance on this page. We ask our team members to consider the factors below in their communication.)

As GitLab matures, we want to continue to foster discussion while evolving our communication guidelines so that topics that are potentially GitLab sensitive are discussed in appropriate forums. This is particularly relevant as team members heavily leverage async modes of communication including merge requests, issues and epics, and in Slack communication.

Words have impact long after they are written, and even when you're communicating internally, the manner in which you speak with one another should be viewed through an external lens. For additional information, please review our [Guidelines for communicating effectively and responsibly through text](#).

Confidentiality levels

At GitLab, we are [public by default](#), but some information is classified as internal or limited access. Please see the [confidentiality levels](#) handbook page for details on this.

Examples of Potentially GitLab Sensitive Topics

1. Team member data (individual performance, start dates, departures)
2. Violations, or potential violations, of policies and/or local rules and regulations
3. Customer or partner information (logos, trademarks, spend)
4. Material nonpublic information

The above examples overlap with the [GitLab's SAFE Framework](#) examples. We recommend you to further review that page for more information and context.

What are the risks?

1. **Legal risk:** These are the risks that arise from regulations and laws that govern GitLab or the market in which it operates. This includes, but is not limited to, content that would compromise a GitLab team member, customer, or user's personal data and/or privacy.
2. **Morale risk:** Raising GitLab sensitive topics that may be misinterpreted without the opportunity to ask clarifying questions can create risk to team culture and/or morale.
3. **PR risk:** Remember that anything you document could ultimately be shared/viewed externally. Consider that a discussion in a public MR or issue is a demonstration of our [values](#) to those outside of GitLab who are looking to learn more about how we collaborate.

We encourage communicating risks to GitLab, its team members, or customers in a synchronous 1:1 setting.

Determining Which Communication Forum To Use

The table below outlines an overview of different communication forums at GitLab, and the considerations team members should think through related to potentially GitLab Sensitive topics when determining which forum to leverage.

Communication Forum	When To Utilize
Use the internal Handbook aligned with the guidelines	When you want to document information for team members that is internal-only , including not public and not limited access
External MR/issue (not confidential)	For discussion and collaboration when there is no risk suspected or identified (directly or later) and it doesn't fall into not public category
Internal MR/ confidential issue	For discussion and collaboration when there is no risk suspected or identified but adhering to things that are internal-only , not public , SAFE Framework/General Communication Guidelines
Your Manager, DRI and/or Legal	For discussion and collaboration when in doubt about potential risk and you want to review if there's a potential risk
DRI	For discussion and collaboration when there's a risk suspected or identified, communicate directly with

Communication Forum**When To Utilize**

	the DRI verbally via Zoom. Examples include issues where Limited Access applies or it covers a change to People Process/policy.
People Group (your People Business Partner or Team Member Relations)	For discussion and collaboration when there's a risk suspected or identified, a policy violation, and/or it's a private matter
GitLab's Whistleblower policy	For flagging a situation that is a violation as set out in the policy

When in doubt, you can reach out to your People Business Partner and/or your leadership team directly.

Organization code names

Please see our [Project names section](#).

Internal communication

Internal communication is any work related communication at a company. Internal Communication includes team member conversations, wider team discussions, or internal announcements to the company or targeted outreach to select internal audiences. At GitLab, everyone can contribute to the effectiveness of Internal Communications to support aspects of GitLab culture, such as intentional transparency and engaging people in open dialogue.

Since we believe that all team members must be [Managers of One](#), most communication is handled by the relevant group, but we know that some communications are more sensitive and contentious than others. In those cases, the DRIs may want to engage the [Internal Communications function](#).

Top tips and best practices

1. All written communication happens in English, even when sent one on one, because sometimes you need to forward an email or chat.
2. Use **asynchronous communication** when possible: merge requests (preferred) or issues. Announcements happen on the appropriate Slack channels and people should be able to do their work without getting interrupted by chat.
3. Discussion in issues or Merge Requests is preferred over everything else. If you need a response urgently, you can Slack someone with a link to your comment on

an issue or merge request, asking them to respond there, however be aware that they still may not see it straight away. See [Slack](#) for more.

4. If you choose to email instead of chat it is OK to send an *internal* email that contains only a short message, similar as you would use in chat.
5. You are not expected to be available all the time. There is no expectation to respond to messages outside of your planned working hours.
6. Sometimes synchronous communication is the better option, but do not default to it. For example, a video call can clear things up quickly when you are blocked. See the [guidelines on video chats](#) for more detail.
7. It is OK to ask as many questions as you have. Please ask them so many people can answer them and so others can benefit from seeing the answer. Use issues or public chat channels (like `#questions`) instead of direct messages or one-on-one emails. If you have researched in the handbook and could not find the answer or need clarity, include the handbook link you were reviewing and state “while looking in the handbook I could not find x,y,z”.
8. If you send a handbook link to someone as an answer to a question, consider adding some context, especially if they are new to GitLab. It’s great that we often have the answer documented, but it’s not always easy to find if you are a new team member.
9. If the answer to a question isn’t documented, please immediately make a merge request to add it to the handbook in a place you have looked for it. It is great for the person who answered the question to see you leading by example to ensure that question only needs to be answered once. A merge request is the best way to say thanks for help.
10. If you mention something (a merge request, issue, commit, webpage, comment, etc.) please include a link to it.
11. All company data should be **shareable** by default. Do not use a local text file, but rather leave comments on an issue.
12. When someone asks something, give back a deadline or that you did it. Answers like: ‘will do’, ‘OK’, ‘it is on my todo list’ are not helpful. If it is small it’s better to spend 2 minutes and do the tasks so the other person can mentally forget about it. If it is large you need to figure out when you’ll do it, by returning that information the other person might decide to solve it in another way if it takes too long.
13. It is OK to bring an issue to someone’s attention with a CC (“cc @user”), but CCs alone are not enough if specific action is needed from someone. The mentioned user may read the issue and take no further action. If you need something, please explicitly communicate your need along with @ mentioning who you need it from.
14. Avoid creating private groups for internal discussions:
 1. It’s disturbing (all users in the group get notified for each message).
 2. It’s not searchable.

3. It's not shareable: there is no way to add people in the group (and this often leads to multiple groups creation).
4. They don't have a subject, so everyone has to remember the topic of each private group based on the participants, or open the group again to read the content.
5. History is lost when leaving the group or [after 90 days](#).
15. It is perfectly fine to create a channel, even for a single customer meeting. These channels should be named "a_-internal" to indicate their "internal" nature (not shared with customers).
16. Use [low-context communications](#) by being explicit in your communications. We are a remote-only company, located all over the world. Provide as much context as possible to avoid confusion. Relatedly, we use [ubiquitous language](#) for communication efficiency.
17. When discussing concepts, be careful not to lean too much into hypotheticals. There is a tipping point in which it decreases value and no longer becomes constructive at helping everyone come into a unified decision.
18. Consult our [tips for better writing](#).

Internal Communication Channel Guidance

What/When	The GitLab Handbook	The Loop	Slack	
Primary Purpose	Transparency around GitLab procedures, documenting information handbook-first	Connect team members with news, information, and stories	Real-time collaboration and quick communications	
Best For	1.) Sharing processes and/or procedures valuable to customers, community, or candidates 2.) Creating external transparency (core values, mission, and vision) 3.) Documenting	1.) Finding/sharing company information, announcements, and events 2.) Learning/sharing programs and initiatives 3.) Accessing personalized	1.) Immediate discussions 2.) Quick questions 3.) Team coordination 4.) Time-sensitive, informal updates 5.) Instantaneous feedback/communication 6.) Channel-specific discussions	1) f a 2) v u E c (v l

What/When	The GitLab Handbook	The Loop	Slack	
	workflows and standardized processes	and relevant content 4.) Team member storytelling, connection, and engagement 5.) Team showcases and wins		
Not For	1.) Time-sensitive updates 2.) Internal communication 4.) Team-specific news 5.) Daily operational matters	1.) Technical documentation 2.) Small audience communication 3.) Highly confidential information. Note: Private Sites is an option for safeguarding access	1.) Detailed documentation 2.) Complex content sharing 3.) Evergreen or long-term Information sharing	
Key Features / Benefits	1.) Version control 2.) Public accessibility 3.) Single source of truth for processes	1.) Customization and personalization 2.) Newsletter, feed, employee generated content 3.) AI powered search and content governance 4.) Fosters Communication, connection, and culture	1.) Collaboration and connection 2.) Integrations 3.) Real-time communication 4.) Productivity and efficiency 5.) Supports projects, teams, and transparency	

Structure content instead of using FAQs

We want to [avoid unstructured content](#) which includes FAQs (Frequently Asked Questions), especially for internal communication.

FAQs tend to take on the voice and concerns of assumed personas. Instead of assuming questions, aim to articulate key facts as statements and use these to structure content under topical headers which aren't questions.

Structured content around GitLab, the product, should be in [GitLab Docs](#) and structured content around GitLab, the company, should be in the handbook; we should not use separate documents or locations to share this information.

Restructuring the content

As an example, let's say your FAQ would have a question like:

Q: I'm not seeing widget X, what should I do? A: If you're not seeing widget X, you can verify if it's enabled or not by going to User Profile → Settings and ensure the checkbox is enabled under Enable widget X

You can reframe it to:

How to enable widget X

You can enable widget X by going to User Profile → Settings and ticking the checkbox next to Enable widget X then clicking on the Save button

FAQs are discouraged elsewhere as well

Content guidelines across the industry support avoiding FAQs:

1. <https://www.plainlanguage.gov/guidelines/web/avoid-faqs/>
2. <https://digital.gov/2015/04/27/are-faqs-still-relevant/>
3. <https://guides.18f.gov/content-guide/our-approach/structure-the-content/>
4. <https://gds.blog.gov.uk/2013/07/25/faqs-why-we-dont-have-them/>
5. <https://thegood.com/insights/faq-pages/>
6. <https://alistapart.com/article/no-more-faqs-create-purposeful-information-for-a-more-effective-user-experi/>

Multimodal communication

Employ multimodal communication to broadcast important decisions. To reach our distributed organization, announce important decisions in the company announcements Slack channel, email the appropriate team email lists, Slack the

appropriate channels, and target 1:1s or other important meetings on the same day, with the same information.

When doing this, create and link to a [single source of truth](#): ideally the [handbook](#), otherwise an epic, issue, or Google Doc. The email or Slack message should not be the source of truth.

When referring to email that recipients should have received, reference the sender and subject of the email so it's easy to find. For example, "You should have received an email from Jane Smith with the subject 'Training Seminar Details'".

Asking "is this known"

If something is behaving strangely on <https://gitlab.com>, it might be a bug. It could also mean that something was changed intentionally.

Please search if the issue has already [been reported](#). If it has not been reported, and you are sure it is a bug, please [file an issue](#).

If you are unsure whether the behavior you experience is a bug, you may ask in the Slack channel [#is-this-known](#). If you know which stage of the DevOps lifecycle is affected, it is also okay to ask in `#s_{stage}`, for example [#s_manage](#).

When you ask:

- Make sure that no-one has experienced this issue before, by checking the channel for previous messages.
- Describe the behavior you are experiencing, this makes it searchable and easier to understand. Different people might look for different things in the same screenshots.
- Asking in a single channel helps discoverability, duplicated efforts and reduces noise in other channels. Please refrain from asking in general purpose channels like [#frontend](#), [#backend](#), [#development](#) or [#questions](#).
- For critical production issues, check if there is a related ongoing incident in [#incidents-dotcom](#). If there is none, please [report an incident](#).

Numbering is for reference, not as a signal

When taking notes in an agenda, in the handbook, or in other documents, keep items numbered so we can refer to Item 3 or 4a. The number is not a signal of the importance or rank of the subject unless explicitly stated to be such. It is just for ease of reference.

Cross link

Linking should not be in one direction. We should go beyond deep-linking to create a richer web of links that can surface content and ensure people consider all pages when making updates. When linking one page to another, try to link back as well. Instead of only linking from Page A to Page B, both link Page A to Page B and link Page B back to Page A. For example, the [Live Doc Meeting](#) section of the All Remote Guide links to the [Live Docs Meetings page](#). The [Live Docs Meetings page](#) links back to the [Live Doc Meeting](#) section of the All Remote Guide.

Acknowledgement Receipts (ACK)

Informal ACKs

In informal acknowledgement scenarios, such as on Slack or on issue comments, it is common practice to use the following:

- Slack emoji reaction of `:ack:` or an `ACK` response ⇒ Acknowledged, or message received
- Eyes 👁 ⇒ I'll check this out | seen | working on it
- Thumbs up 👍 ⇒ good idea
- White checkmark ✅ ⇒ task is complete or done
- Heart ❤ = expression of gratitude or appreciation
- cc @mentions ⇒ if someone needs to see a message

Formal ACKs

In order to effectively communicate an important change to hundreds of distributed employees, we occasionally use an ACK process.

To prevent overuse, this should only be used by a member of the exec team. Anyone may ask an exec to sponsor one.

As a guideline, we'd expect no more than one per quarter to be sent out. Too many ACKs lose power.

To initiate an ACK process:

1. Clone the form from the [ACK template](#) and fill it out.
 1. Link to MRs and Handbook pages instead of duplicating your content in the form. [Why handbook first?](#)
2. Ask People Ops to pull a report from Workday with the column headers First Name, Last Name, Job Title, Department, Manager, and Work Email. Double check it and turn the emails into a comma-delimited string with an excel formula like this:


```
=TEXTJOIN(", ", true, Sheet1!E2:E432)
```

3. Send the form and expect to get 50% of the responses in the first 24 hours. To get the rest:
 1. Post in common Slack channels.
 2. Add to staff meeting agendas.
 3. Suggest to team managers to post to their team Slack channels, ask for explicit :ack: and pin to the channel until everyone responds.
 4. Lastly, reach out 1-on-1 to stragglers while being respectful of vacation time.

Say thanks

As we continue to build on [inclusion](#), recognition is a key and transformative tactic. Thanking team members provides an opportunity for them to be recognized for their contributions, influences engagement behavior, and acknowledges to team members their work is seen. By saying thanks, you are contributing to and supporting the value of [DIB](#).

1. Thank people that did a great job in our #thanks Slack channel. Almost [everyone in the company is active in this channel](#) so please don't be shy.
2. Consider other channels where recognition can be acknowledged: team meetings, issues, company calls, 1-1 meetings, etc.
3. If someone is a team member just @-mention them, if multiple people were working on something try @-mentioning each person.
4. When announcing a completed project, list the key contributors.
5. Please be as timely as possible with your recognition.
6. If possible please include a link with your thanks that points to the subject matter that you are giving thanks for, for example a link to a merge request.
7. We generally avoid recognizing people for working outside of working hours to minimize the pressure to do so. At times, team members may decide to work outside their core working hours to complete a task or attend an event. When team members work outside their core hours to deliver [results](#), we acknowledge their flexibility and contribution [transparently](#). It is appropriate to thank team members who flex their schedule to meet GitLab goals whether that's resolving a customer emergency, accommodating global team meetings, or delivering on a critical deadline.
8. Do not thank the CEO or other executives for something that the company paid for, thank GitLab instead.
9. To thank someone who is not a team member, [nominate them as a Notable Contributor](#).
10. Understand that everyone doesn't need or want recognition. Once this is advised, please respect when they don't.

Values emoji

Add Values emoji reactions to thank you messages in the [#thanks_slack_channel](#) or feel free to use them in GitLab.com, other slack channels and social media, when you see alignment to our values: [GitLab's values](#).

Emoji	Custom values emoji	Meaning
:handshake:	:collaboration-value:	Collaboration
:chart_with_upwards_trend:	:results-value:	Results for Customers
:stopwatch:	:efficiency-value:	Efficiency
:globe_with_meridians:	:diversity-value:	Diversity Inclusion and Belonging
:footprints:	:iteration-value:	Iteration
:eye:	:transparency-value:	Transparency



As a second iteration, we have [added these custom emoji to GitLab](#) to enable tanuki values reactions in issues, epics, and MRs within the `gitlab-com` and `gitlab-org` groups.

As a later iteration, we will begin tracking the number of emoji reactions for each value through the Reacji API and update this page with our findings!

Indicating availability

Indicate your availability by updating your own calendar using Google's ["out of office"](#) feature and include the dates you plan to be away in your automated response. Note that this feature will automatically decline any meeting invitations during the time frame you select.

1. Put your planned away time including holidays, vacation, travel time, and other leave in your own calendar. Please see [Communicating your time off](#) for more.
2. Set your working hours in your Google Calendar settings.

3. Utilize Time Off by Deel to keep other GitLab team members aware of your [planned time away](#) within Slack.

Informal communication

[Informal communication](#) is made up of interactions between co-workers that are unofficial in nature and focus on building social relationships outside of the normal hierarchy of a typical business structure.

In other words, it's what happens when we get to know each other and talk about anything other than work.

Informal communication is a [vital part of GitLab culture](#), and we've listed [20+ ways to engage](#).

Not sure where to go?

If there is something that you want to discuss, but you do not feel that it is a reasonable option to discuss with either your manager or CEO, then you can reach out to any of the other [C-level GitLab team members](#).

Start with a Merge Request

When possible, it's best practice to start a discussion with a [Merge Request \(MR\)](#) instead of an issue. An MR is associated with a specific change that is proposed and transparent for everyone to review and openly discuss. The nature of MRs facilitate discussions around a proposed solution to a problem that is actionable. An MR is actionable, while an issue will take longer to take action on.

1. Always **open** an MR for things you are suggesting and/or proposing. Whether something is not working right or we are iterating on a new internal process, it is worth opening a merge request with the minimal valuable change instead of opening an issue encouraging open feedback on the problem without proposing any specific change directly. Remember, an MR also invites discussion, but it's specific to the proposed change which facilitates focused decision.
2. Never ask someone to create an issue when they can default to the merge request.
3. Starting with a Merge Request is part of [Handbook First](#) and helps ensure the handbook is up-to-date when a decision is made. It is also how we make it possible for [Everyone to Contribute](#). This is true, not just for updating the handbook but for updating all things.
4. Merge Requests, by default, are **non-confidential**. However, for [things that are not public by default](#) please open a confidential issue with suggestions to specific

- changes that you are proposing. The ability to create [Confidential Merge Requests](#) is also available. When possible, consider not including sensitive information so the wider community can contribute.
5. Not every solution will solve the problem at hand. Keep discussions focused by **defining the problem first** and **explaining your rationale** behind the [Minimal Valuable Change \(MVC\)](#) proposed in the MR.
 6. Be proactive and consistent with communication on discussions that have external stakeholders such as customers. It's important to keep communication flowing to keep everyone up to date. MRs can appear stale if there aren't recent discussions and no clear definition on when another update will be provided, based on feedback. This leaves those subscribed in the dark, causing unnecessary surprise if something ends up delayed and suddenly jumps to the next milestone. It is important that MRs are closed in a timely manner through approving or rejecting the open requests.
 7. Have a **bias for action** and [do not aim for consensus](#). Every MR is a [proposal](#), if an MRs author isn't responsive take ownership of it and complete it. Some improvement is better than none.
 8. **Cross link** issues or other MRs with related conversations. E.g. if there's a Zendesk ticket that caused you to create a GitLab.com MR, make sure to document the MR link in the Zendesk ticket and vice versa. And when approving or rejecting the MR, include reason or response from Zendesk. Put the link at the top of each MR's description with a short mention of the relationship (Report, Dependency, etc.) and use one as the central one and ideally close the alternate if duplicate.
 1. When providing links to specific lines of code relevant to the MR, **always use a permalink** (a link to a specific commit for the file). This ensures that the reference is still valid if the file changes. For more information, see [Link to specific lines of code](#).
 9. If submitting a change for a feature, **update the description with the final conclusions** (Why an MR was rejected or why it was approved). This makes it much easier to see the current state of an issue for everyone involved in the implementation and prevents confusion and discussion later on.
 10. Submit the [smallest viable and valuable thing](#). When proposing a change, submit the smallest reasonable commit, put suggestions for other enhancements in separate issues/MRs and link them. An MR can start off as only a [problem description and TODO comments](#). If you're new to GitLab and are writing documentation or instructions, submit your first merge request for at most 20 lines.
 11. Do not leave MRs open for a long time. MRs should be **actionable** – stakeholders should have a clear understanding of what changed and what they are ultimately approving or rejecting.
 12. Make a conscious effort to **prioritize** your work. The priority of items depends on multiple factors: Is someone waiting for the answer? What is the impact if you

delay it? How many people does it affect, etc.? This is detailed in [Engineering Work flow](#).

13. When submitting a MVC, **ask for feedback** from your peers. For example, if you're a designer and you propose a design, ping a fellow designer to review your work. If they suggest changes, you get the opportunity to improve your design and propose an alternative MR. This promotes collaboration and advances everyone's skills.
14. Respond to comments within a **threaded discussion**. If there isn't a discussion thread yet, you can use the [Reply to comment](#) button from the comments to create one. This will prevent comments from containing many interweaved discussions with responses that are hard to follow.
15. If your comment or answer contains separate topics, write separate comments for each, so others can address topics independently using the [Reply to comment](#) button.
16. If you have received any feedback or questions on your MR, try to acknowledge comments as [that's how we ensure we create an environment of belonging for all team members](#). Merging your MR as-is without giving an answer or any response makes the commenters feel their opinions are unheard. If you are the [Directly Responsible Individual](#) (DRI) who does not have to make a fast decision, you can choose not to change your MR, but you should acknowledge the comments or feedback, consider if they warrant a change to your MR, and [say why, not just what](#). If there are many comments, you can choose to summarize key feedback areas and share your response at a high level. We appreciate that [if you force a DRI to explain too much, you'll create incentives to ship projects under the radar. The fear of falling into a perpetual loop of explaining can derail a DRI, and cause people to defer rather than working with a bias for action](#). This is something we want to avoid. When fast decisions are needed, [we'll have to accept that people listened to us but don't owe us an explanation to have fast decisions based on everyone's input](#). The goals are to be transparent and collaborative—not to lose efficiency. Not everyone will agree, but we expect all people to [disagree, commit, and disagree](#).
17. For GitLab, the product merge request guidelines are in the [Contribution guide](#) and code review guidelines for reviewers and maintainers are described in our [Code Review Guidelines](#).
18. Even when something is not done, share it internally so people can comment early and prevent rework.
19. Create a **Draft** merge request to prevent an accidental early merge. Only use Draft when merging it would **make things worse**, which should rarely be the case when contributing to the handbook. Most merge requests that are in progress don't make things worse. In this case, do not use Draft; if someone merges it earlier than you expected just create a new merge request for additional items. Never ask

- someone to do a final review or merge something that still has Draft status. At that point you should be convinced it is good enough to go out.
20. If any follow-up actions are required on the issue after the merge request is merged (like reporting back to any customers or writing documentation), avoid auto-closing the issue.
 21. If a project requires multiple approvals to accept your MR, feel free to assign multiple reviewers concurrently. This way the earliest available reviewer can start right away rather than being blocked by the preceding reviewer.
 22. If the MR involved gets a lot of comments, you can turn it into a [Manager Mention MR](#).
 23. Consider recording a concise video or audio file outlining the merge request and uploading it to the GitLab [Unfiltered channel](#) on YouTube. This will make content more accessible, prevent future confusion, allow for multitasking (e.g. cooking dinner and listening to the video), and increase participation for folks who digest audio information better than visual.

Scaling Merge Requests through “Manager Mention MRs” (formerly Consolidated MRs)

Some merge requests that involve a big decision or change tend to collect a large amount of feedback. As GitLab grows in size, it is unrealistic for a single person to respond to potentially hundreds of comments. To remain efficient in these MRs and to make it scalable, it is important for the DRI to receive a clear signal of input that is shared on the merge request. Some MRs may be marked as “Manager Mention MRs” by clearly designating them as such at the beginning of the MR description with the following code block:

```
### Manager Mention MR

This MR is a [Manager Mention MR](/handbook/communication/#scaling-merge-requests-t
```

Additionally, add the ~"Manager Mention MR" label to the merge request. This will make future analytics on Manager Mention merge requests more easily identifiable. It also enables managers to subscribe to the label to be notified when an MR has elected the Manager Mention MR designation.

We tried Manager Mention MR's for the first time in a recent announcement (2021-03-03) but this did not work well and we are working on making it better. We're starting with a more thoughtful and transparent process in our communications cadence and approach going forward, including all directs and people managers getting a few days'

notice before important company-wide changes are announced to all team members. This will allow all directs and people managers to feel more enabled and better understand the why behind big changes in order to scale communication to team members.

For all managers: It is important to ground yourself in the contents of the changes before the announcement goes live to all team members. If a team member tags you in a Manager Mention MR, it is your role to respond candidly and thoughtfully to their question or comment. If the line of questioning in the Manager Mention MR gets out of your depth, ask the DRI to help answer. If a team member comments without a manager tagged, the comment will be closed with a link to this handbook section or closed without comment. In a situation where a team member leaves a wildly inappropriate comment in the Manager Mention MR, you should feel empowered to delete comment and talk to your team member 1:1. Consider subscribing to the label `~"Manager Mention MR"` to be notified when MRs transition to this designation.

- **What not to do:** Not communicate to team members about company-wide changes. Ignoring team member questions, whether that's in a 1:1 or Manager Mention MR.
- **What to do:** If one of your team members has a suggestion, solution or sees an issue, see if talking through the communication will answer any of their questions. Team members are also allowed to bring forward their ideas in the Manager Mention MR addressed to you, we want everyone to contribute. As a manager, you will be expected to communicate changes to your teams and also be present to answer any team member questions, whether that's in a 1:1 or Manager Mention MR. As a manager, it is part of your role to understand and own change management for your team and properly triage the process and expectations. Consider a synchronous call with the team member for further context, make a suggestion, link to additional context, delete any unnecessary comments from their team, or escalate to the author of the MR. Ensure that the comments of your reports you interact with were made after the Manager Mention label was added.

For team members: Check if the MR you are about to comment on has the `~"Manager Mention MR"` label. Check each time as the label may have been added since you last commented. When leaving a comment in a Manager Mention MR, frame the comment as a question or suggestion to your manager directly, and not anyone else, including the DRI. We do this to scale communication, as it is unsustainable for the DRI to answer every question.

- **What not to do:** Leaving a comment and CC'ing your manager at the end of your post, but not addressing your manager directly.
- **What to do:** Address your manager directly at the beginning of your message as your comments should be addressed to your manager. If you have a suggestion, solution or see an issue with a big change, you can also bring it up directly to your

manager. The MR is not a poll. Give suggestions on how to improve and try to find data that helps support your argument or change in the MR.

MRs should not start out as a Manager Mention MR as we prefer [communication to be direct](#). They should only be designated as such after the number of comments on them grows to a level that is unsustainable for the DRI. An exception to this is compensation changes and other company-wide announcements that can be sensitive/contentious in nature since they have historically generated many comments.

When an MR is changed to be `Manager Mention`, the person making this change should add a comment stating this so that everyone tracking the MR can be informed.

Issues

Issues are valuable when there isn't a specific code change that is being proposed, such as:

- Crafting a research proposal to validate a problem or solution
- Ideating on designs in order to solve a particular problem
- Breaking down implementation tasks in order to deliver a solution iteratively
- Tracking progress of particular tasks, especially when an issue board is needed

When utilizing issues, it is still important to maintain focus by defining a single specific topic of discussion and the desired outcome that would result in the resolution of the issue. Issues should not be open-ended or go stale due to lack of resolution. For example, a team member may open an issue to track the progress of a blog post with associated to-do items that need to be completed by a certain date (e.g. first draft, peer review, publish). Once the specific items are completed, the issue can successfully be closed.

Below are a few things to remember when creating issues:

1. When **closing** an issue leave a comment explaining why you are closing the issue and what the MVC outcome was of the discussion (if it was implemented or not).
2. We keep our **promises** and do not make external promises without internal agreement.
3. Be proactive and consistent with communication on discussions that have external stakeholders such as customers. It's important to keep communication flowing to keep everyone up to date. Issues can appear stale if there aren't recent discussions and no clear definition on when another update will be provided, based on feedback. This leaves those subscribed in the dark, causing unnecessary surprise if something ends up delayed and suddenly jumps to the next milestone. It is important that issues are closed in a timely manner. One way of doing this is having the current assignee set a due date for when they will

provide another update. This can be days or weeks ahead depending on the situation, prioritization, and available capacity that we may have.

Pro Tip: When creating a Merge Request you can add `closes: #[insert issue number here]` and when the Merge Request is merged, the issue will automatically close. You can see an example of this [here](#). **Note:** [Automatic issue closing](#) is disabled on some projects.

1. If a user suggests an enhancement, try and find an existing issue that addresses their concern, or create a new one. Ask if they'd like to elaborate on their idea in an issue to help define the first MVC via a subsequent MR.
2. **Cross link** issues or MRs with related conversations. Another example is to add "Report: " lines to the issue description with links to relevant issues and feature requests. When done, add a comment to relevant issues (and close them if you are responsible for reporting back, or reassign if you are not). This prevents internal confusion and us failing to report back to the reporters.
3. When cross-linking issues or MRs, include a preview of the content you are linking, to facilitate [low-context communication](#):
 1. Good: this would cause performance issue similar to #123456 . The reader has full information on first read and can refer to the link for more.
 2. Avoid: this would cause issue similar to #123456 . The reader needs to click the link and find the relevant information among other discussion threads, before switching back to the original discussion.
4. When providing links to specific lines of code relevant to the issue, **always use a permalink** (a link to a specific commit for the file). This ensures that the reference is still valid if the file changes. For more information, see [Link to specific lines of code](#).
5. Prioritize your work on issues in the current [milestone](#).
6. Use the public issue trackers on GitLab.com for everything since [we work out in the open](#). Issue trackers that can be found on the relevant page in the handbook and in the projects under [the gitlab-com group](#).
7. Assign an issue to yourself as soon as you start to work on it, but not before that time. If you complete part of an issue and need someone else to take the next step, **re-assign** the issue to that person.
8. Ensure the issue **title** states what the desired outcome should be. For instance, for bugs make sure the issue states the desired result, not the current behavior.
9. **Regularly update** the issue description with the latest information and its current status, especially when important decisions were made during the discussion. The issue description should be the **single source of truth**.
10. If you want someone to review an issue, do not assign them to it. Instead, @-mention them in an issue comment. Being assigned to an issue is a signal that the

assignee should or intends to work on it. So you should not assign someone to an issue and misrepresent this with a false signal.

11. If you'd like to inform someone about an issue or assign a task to them, do so via an issue comment, not only by adding them to the description. The to-do item generated when you mention someone in an issue description provides little context for the action you're requesting. But using a comment to explicitly inform someone of the action you'd like them to take ensures that when they read the associated to-do item they won't need to read the entire issue to gather the context they need to complete the work.
12. Do not close an issue until it is [done](#). It's okay to explicitly ask if everyone is on board and in agreement on how to move forward, whether to iterate, close the open issue, or create a subsequent MR to implement a MVC.
13. Once a feature is [done](#), update the description to add a link to the corresponding documentation. When using a Search Engine, issues often appear before documentation pages, which makes it harder to find the relevant information about the feature.
14. Write issues so that they exclude private information. This way, the issue can be public. Only use confidential issues, if the issue must contain [non-public information](#). **Note:** Confidential issues are [accessible to all members of the project with Reporter access and above](#). You may consider using a Google Doc for items that require a stricter level of confidentiality.
15. If the content within a public issue transitions to become what is deemed confidential [non-public information](#), the issue may be made confidential.
16. If the content of a public issue draws comments that are deemed in violation of our [code of conduct](#) the issue may be locked and may [undergo moderation](#).

How to make a company wide announcement

1. Consider the subject and the audience. Questions you might want to ask yourself: Is this relevant to all team members globally? Is this something important, urgent and high priority? Is there a better place for this communication, such as a more informal Slack channel?
2. Keep it simple, brief and summarize what is important. Cover the 5 W's. What, Why, Who, When, Where (you can also add How, if required as a call to action). The majority of information should still be in the Handbook which you include links to.
3. Common company wide announcements include (but are not limited to): organization changes, policy iterations, requests to participate in a company survey, unveiling the next GitLab Contribute location, codebase migrations, process improvement and security/safety announcements.
4. Remember Handbook First. When you announce anything, include links to the respective Handbook pages for more information. Consider adding link to an Issue if the information is not public yet.

5. Optional AMA. If desired and appropriate, offer a company wide Zoom call to host an AMA (Ask Me Anything). Oftentimes, questions can be managed within the Discussion tab of a GitLab Issue or Merge Request. For broad announcements, such as registration opening for GitLab Contribute, an AMA may be better suited for a large volume of inquiries. To schedule a company wide call, please make a request in the `#people-connect` Slack channel, and include a Google Doc in the invite for questions.
6. Remember we are a global company with significant time-zone differences. Unless there is a reason to do otherwise, ensure any time-sensitive calls to action or announcements are made when the whole company has enough time to action. Consider different timezones, non-linear work days, and PTO. Announcements should be made ideally 72 hours (at minimum 24 hours) in advance of a due date. This is to prevent APAC/EMEA team members missing important announcements posted outside their normal working hours. Sometimes a late announcement is better than none at all, and acknowledging those who will miss it might be a kind gesture, such as “Apologies to our APAC/EMEA friends for the late notice”.

Posting in #company-fyi

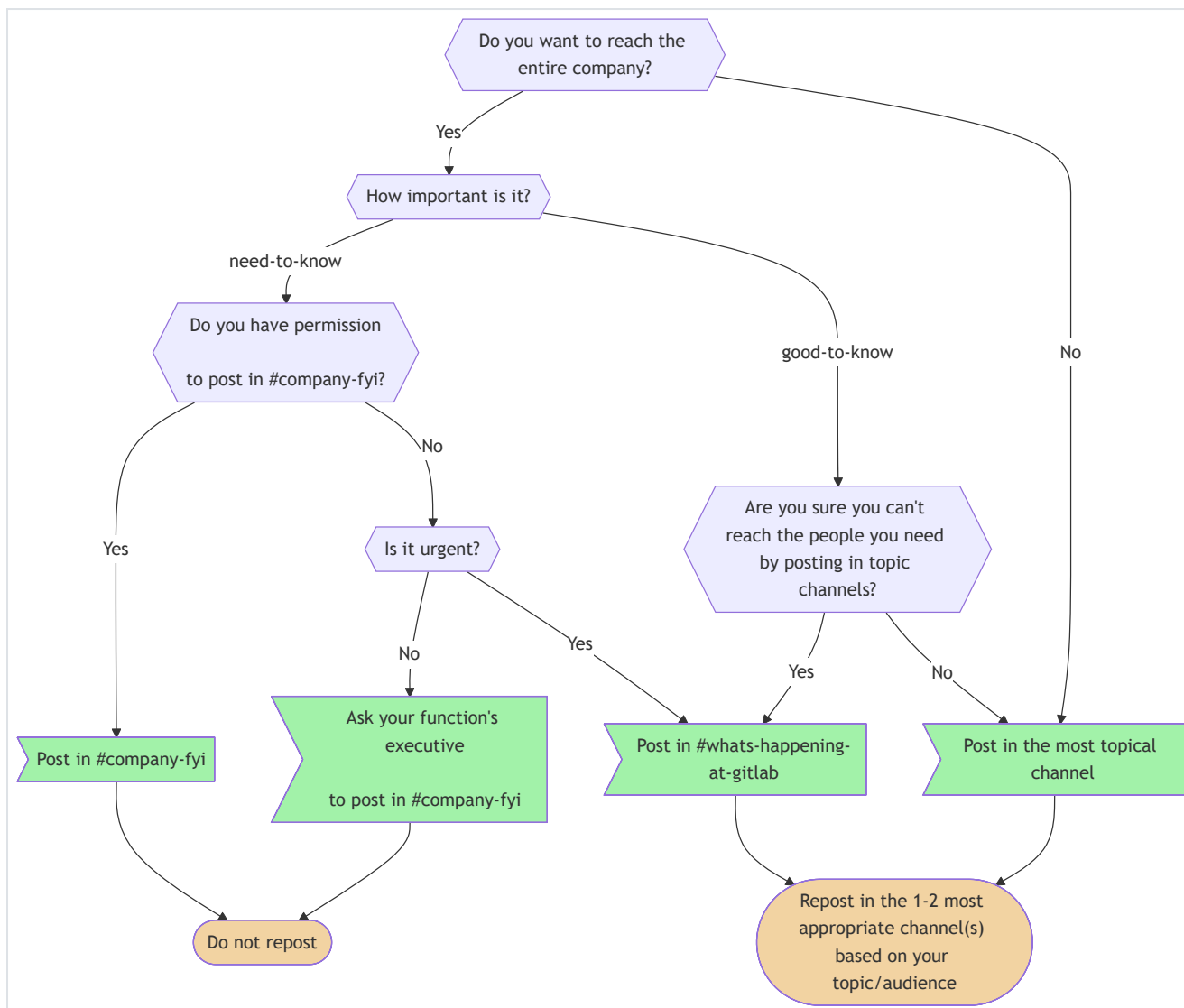
Our company-wide announcements channel is **#company-fyi**. It is an **announcement only** channel, meaning that communications need to be approved before they can be posted. To minimize noise, announcements made in `#company-fyi` should not be duplicated in `#whats-happening-at-gitlab`. Be mindful of the [attention economy](#).

In order to post or have a message posted in `#company-fyi`, please reach out to [the internal communications team](#) or your function’s executive who can approve the message and post it.

Examples of what **should not** go in **#company-fyi** (as per new group guidelines):

- Competition prize winner announcements
- Org change or new team member announcements (unless they are E-group)
- Promotion of an optional non-company-wide internal event
- Announcement that directly impacts less than 75% of team members
- Actions required from team members is not critical or timely

The above should all go in the new #whats-happening-at-GitLab channel (formerly the `#company-announcement` channel).



Posting in #whats-happening-at-gitlab

Due to the volume of posts in the Slack channel, we recommend that you do not use #whats-happening-at-gitlab as a sole location for important announcements as information might get lost or muted. Examples of important items include but are not limited to:

- Anything involving GitLab team member policy, such as changes in benefits, laws, review cycles, etc.
- Urgent matters that can't wait for #company-fyi but still need to be communicated to everyone such as service outages or last minute event changes

Meetings

Common meeting problems

Meetings are incredibly expensive since they require synchronous time. The most common meeting problems can all be addressed by following the above guidelines

around scheduling meetings. Some of the most common meetings problems are outlined below:

Problem	Solution
Present instead of Q&A	Pre-record presentations on YouTube, so meetings are only Q&A
Meetings set up for or default to brainstorming	People should default to making thoughtful proposals async and building upon them in meetings, if needed
No agenda with edit rights for everyone	Ensure that every meeting has an agenda and is available for everyone to edit
People are late to meetings or don't have time to use the restroom between meetings	Use Speedy Meetings to give people breathing space before their next meeting

Everyone is responsible for notes

If folks are involved in a meeting and have capacity to do so, they should take notes using GitLab's [Live Doc Meetings principles](#). This is important, because:

1. Meetings at GitLab should have notes (for a single source of truth and to enable async participation among other reasons)
2. In the absence of this joint commitment to note-taking, this is the type of work that is likely to fall disproportionately to [underrepresented groups](#). This is not in line with our [diversity, inclusion, and belonging value](#).

It may look like a few people are already taking notes, do not see this as a deterrent for helping. Initial note takers may be first to show up and then see it as their responsibility to continue if no one else is stepping in.

While meetings recordings are helpful, written notes are more efficient to read and offer greater opportunities for async engagement. Takes notes even when a meeting is being recorded.

Engaging EBAs in note-taking

[GitLab Executive Business Administrators](#) sometimes support teams by taking notes. Since note taking takes time away from their other activities and can often be done by other folks, consider the following before engaging an EBA in a meeting solely for note taking purposes.

1. Can the folks already in this meeting cover note taking responsibilities, or is there a reason to engage an EBA in this capacity?
2. Can other folks be identified ahead of time or at the start of the meeting to ensure adequate coverage?
3. How does this stack against the EBAs other priorities? You can check directly with the EBA or their manager.

Smart note taking in meetings

Note taking helps us to work asynchronously. Team members can add thoughts to an agenda in advance of a meeting and understand what was discussed if they cannot attend. It also offers a record of discussion.

Consider the following best practice when taking notes in meetings:

1. If at the start of the meeting, it does not look like all team members will contribute to note-taking, identify a set of note-takers who will be responsible for this activity within the meeting
2. Utilize Zoom's AI Companion to create and/or share a meeting summary. Meeting Summary with Zoom AI Companion uses AI technology, which may include third-party models, and allows meeting hosts to initiate an AI-generated summary of their meeting. When the host enables this feature in a meeting, participants may automatically receive a summary after the meeting ends, if the host chose to share it with them.
3. Note-taking can be a lot for a single person to stay on top of—especially when there is a fast moving conversation with many speakers. Team members should still feel empowered to contribute by helping with notes as needed, even if there is someone assigned.
 - Another good reference is the CEO Shadow [handbook page tips section](#), specifically It's helpful if shadow one takes notes as the first speaker is talking, then shadow two starts when the next speaker continues the conversation. Shadow one can pick up note taking again when the next speaker contributes. By alternating this way, the shadows are better able to keep up with all the participants in the conversation. This advice can be used in general meetings as well.
4. Ask others to scribe answers in real-time to allow the person who asked the question to focus on the answer. Touch up the answer when the conversation has moved on to something less relevant.
5. It can be hard to keep up with the dialog and capture quality notes when there's fast back and forth conversation. Lead by example and write when you're not talking, expect others will write when you're talking.

6. Focus on noting speakers and their key points over capturing all words said. Extensive note-taking should not happen at the expense of correct note-taking.
7. [Write down your questions](#) in the agenda before vocalizing. Always ask people to vocalize their questions to provide the most detailed context and for audio-only playbacks.
8. Use discretion in taking notes if sensitive topics are being discussed. For example, do not take notes on [not-public information](#) if the agenda may be available to an audience who should not be privy to this information.
9. If someone requests folks to stop taking notes, stop for the duration of the discussion unless there is verbal confirmation that note-taking should resume. Ask for the confirmation before typing before you resume note-taking.
10. At the end of the meeting, clearly capture key takeaways, next steps, and [DRIs](#).

If you have any questions about what may or may not be a sensitive topic, please refer to our SAFE Framework or reach out via the `#safe` Slack channel

Few meetings with presentations

[Presenting](#) during meetings requires valuable synchronous time. Instead, recorded presentations make content accessible, prevent confusion, and increase participation for team members that prefer consuming content [asynchronously](#). Remember it is not required to have a presentation or have a pre-recorded presentation.

In the video below, GitLab co-founder, Sid Sijbrandij, explains why there are no presentations in most meetings.

No Presenting in Meetings - CEO Handbook Learning Session

GitLab Unfiltered



Watch on

Pre-recorded presentations enable:

- Allows time for Q&A, which enables attendees to have their questions answered without running out of time.
- Reinforces GitLab's [Bias Towards Asynchronous Communication](#) because it allows a distributed team to consume the presentation asynchronously.
- Strengthens [self-service and self-learning](#) by maximizing the meeting time's efficiency to ensure that team members have their voices heard during the Q&A.
- Standardizes the approach to meetings across the organization.
- Includes transcripts that can boost content value, help team members focus, and increase accessibility.
- Flexibility in viewing using rewind and playback speed adjustments.
- Encourages and enables greater participation from [neurodiverse team members](#) who might take added time to process and reflect before asking questions.
- Allows for selective watching of presentation material for a certain period.

There are times when presenting during a meeting is needed. This may occur when adding more context to a specific topic on slides. If this is the case, consider the following:

- A presentation, with optional attendance and mandatory recording. This will allow clarifying questions to be asked and answered efficiently and enables team members to watch async.
- Include an async Q&A doc for team members who did not attend the presentation.
- Ensure the async Q&A doc is linked in the YouTube description.

Best Practices for Pre-Recorded Presentations

1. Use Zoom to create a pre-recorded video presentation.
2. Post the recording to the GitLab [Unfiltered YouTube channel](#) and attach it to the meeting agenda.
3. At least 24 hours in advance of the meeting, announce in Slack Channels that the meeting has a pre-recorded video, and all attendees are advised to watch beforehand.

Framework for meetings with presentations

While most meetings should not have presentations, there are a few exceptions. Specifically, we may use synchronous touch points in meetings with large numbers of folks. These tend to be meetings used for building team cohesion and alignment. For example, [GitLab Assembly](#) or the [Functional Leaders Meeting](#).

GitLab has the following meeting framework for determining which meetings should have presentations:

Presentation Approach	Types of meetings with few participants	Types of meeting
No presentations (async prep)	Most meetings	AMA
Presentations	These meetings should not happen	Assembly and ot

Meeting introduction guidelines

Introductions can be helpful during some external meetings, such as executive sales calls. In those meetings, use these guidelines:

1. Agree ahead of time to do introductions so everyone is prepared for it.
2. Create a list of people with their roles in a shared agenda and use that for the introduction sequence.
3. Each person should introduce themselves so that everyone can see that person on Zoom.
4. The person introducing themselves hands it over to the next person in the agenda. Make sure you're never screen sharing when people are introducing themselves.

Scheduling meetings

1. Every scheduled meeting should either have a Google Presentation (for example for functional updates that don't require participation) or a Google Doc (for most meetings) linked. If it is a Google Doc it should have an [agenda](#), including any preparation materials (can be a presentation). For a step-by-step guide on GitLab meeting best practices, visit our [Live Doc Meetings page](#).
 - No agenda is required for [coffee chats](#). Note that only meetings that are primarily social in nature should be labeled as a coffee chat in the calendar invite.
 - A suggested format for 1:1 agendas can be found on the [1:1 leadership page](#).
2. For team members working in AMER timezones who regularly collaborate with EMEA team members:
 - If all meeting attendees are located in AMER timezones, the meeting should be scheduled outside of the [PST morning block](#).
 - The PST morning block should be reserved for [cross-regional collaboration](#) with team members whose timezones make it harder for them to meet later in the day.

3. For team members working in AMER timezones who regularly collaborate with APAC team members:
 - If all meeting attendees are located in AMER timezones, the meeting should be scheduled outside of the [PST afternoon block](#).
 - The PST afternoon block should be reserved for [cross-regional collaboration](#) with team members whose timezones make it harder for them to meet earlier in the day.
 - Due to the number of timezones covered in APAC, the [PST afternoon block](#) will only overlap with the Eastern most APAC countries
4. If you want to ask GitLab team members if they are available for an event please send a calendar invite with Google Calendar using your Google GitLab account to their Google GitLab account. When you add a GitLab team member as a “Guest” in Google Calendar, you can click the See Guest Availability button to check availability and find a time on their calendar. These calendar invites will automatically show up on all parties calendars even when the email is not opened. It is an easier way to ensure everyone has visibility to the meeting and member’s status. Please respond quickly to invites so people can make necessary plans.
5. If you want to check if a team member is available for an outside meeting, create a calendar appointment and invite the team member only after they respond yes. Then invite outside people.
6. When scheduling a call with multiple people, invite them using a Google Calendar that is your own, or one specific to the people joining, so the calendar item doesn’t unnecessarily appear on other people’s calendars.
7. If you want to move a meeting just move the calendar appointment instead of reaching out via other channels. Note the change at the top of the description.
8. Please click ‘Guests can modify event’ so people can update the time in the calendar instead of having to reach out via other channels. You can configure this to be checked by default under [Event Settings](#).)
9. When scheduling a meeting we value people’s time and prefer the “speedy meetings” [setting in our Google Calendar](#). This gives us meetings of, for example, 25 or 50 minutes leaving some time to:
 - Write notes and reflect
 - Respond to urgent messages
 - Take a [bio break](#)
 - Stretch your legs
 - Grab a snack
10. When scheduling a meeting, please try to have it start at :00 (hour) or :30 (mid-hour) to leave common start times available for other meetings on your attendees’ calendars. Meetings should be for the time needed, so if you need 15 minutes just book that.

11. When creating a calendar event that will be used company wide, [please place it on the GitLab Team Meetings Calendar](#). That way the event is easily located by all individuals.
12. When you need to cancel a meeting, make sure to delete/decline the meeting and choose the option **Delete & update guests** to make sure everyone knows you can't attend and don't wait for you.
13. If you want to schedule a meeting with a person not on the team please use [Calendly](#). Use Google Calendar directly if scheduling with a GitLab team member.
14. **Materials Review** are scheduled as all day not busy events as a reminder three days before the scheduled call.
15. When scheduling recurring meetings, consider using **(UTC+00:00) Coordinated Universal Time** as the meeting time zone, so that the meeting time does not change for others when your local time zone changes.

Cross-regional Working Hours Recommendations

If you're scheduling a meeting across multiple regions, consider using the time ranges below to respect common working hours.

The suggested times are organized by the regions that you're trying to accommodate. Each suggested window is shown in the local time zone. For example, if your meeting includes team members in EMEA and AMER, you could consider scheduling from 8:00 AM to 10:00 AM Pacific Time.

Note: Time zone offsets change throughout the year due to Daylight Savings Time, Summer Time, and similar time changes, so these suggested times may be less convenient at different times of year.

EMEA/AMER

Timezone	Start	End
London/Lisbon	04:00 PM	06:00 PM
Paris/Rome	05:00 PM	07:00 PM
Istanbul/Tel Aviv	06:00 PM	08:00 PM
Abu Dhabi	07:00 PM	09:00 PM
Mumbai	08:30 PM	09:30 PM
Pacific Time	08:00 AM	10:00 AM

Timezone	Start	End
Eastern Time	11:00 AM	01:00 PM

APAC/AMER

Timezone	Start	End
Sydney	09:00 AM	11:00 AM
Tokyo	08:00 AM	10:00 AM
Hong Kong	07:00 AM	09:00 AM
Ho Chi Minh	07:00 AM	08:00 AM
Pacific Time	04:00 PM	06:00 PM
Eastern Time	07:00 PM	09:00 PM

EMEA/APAC

Timezone	Start	End
Lisbon/Dublin	08:00 AM	10:00 AM
Paris/Rome	09:00 AM	11:00 AM
Istanbul/Tel Aviv	10:00 AM	12:00 PM
Mumbai	12:30 PM	02:30 PM
Ho Chi Minh	02:00 PM	04:00 PM
Hong Kong	03:00 PM	05:00 PM
Sydney	05:00 PM	07:00 PM

Multi-session meeting naming

When scheduling meetings with two or more sessions (usually when trying ensure worldwide coverage for all team members), name them after the topic, appended with a session number based on the order they show up in the calendar. Team members will see the meeting invites in their email or calendar in relation to their local time zone and can decide for themselves which session to attend, based on their working hours.

Avoid:

- Terms like `friendly` or `early / late`, as these terms are overly subjective. An early meeting for one team member might seem late for someone in a different time zone. Or a west coast AMER meeting might seem “APAC friendly”, but not to someone in western APAC who is still asleep when the meeting starts.
- Using `AMER`, `EMEA`, `APAC`, or `only` unless the meeting is specifically targeting members of that time zone. These terms give the impression that only team members from those timezones are welcome, when people from any timezone with any working style are welcome.

For example:

Scheduled time	Preferred	Avoid
07:00:00 UTC	“All members meeting - Session 1”	“All members meeting - EMEA/AMER”
15:00:00 UTC	“All members meeting - Session 2”	“All members meeting - AMER/APAC friendly”
23:00:00 UTC	“All members meeting - Session 3”	“All members meeting - APAC/EMEA only”

Video calls

1. Use video calls if you find yourself going back and forth in an issue/via email or over chat. Guideline: if you have gone **back and forth 3 times**, it's time for a video call.
2. Sometimes it's better to *not* have a video call. Consider these tradeoffs:
 - It is difficult (or impossible) to multi-task in a video call.
 - It may be more efficient to have an async conversation in an issue, depending on the topic.
 - A video call is limited in time: A conversation in an issue can start or stop at any time, whenever there's interest. It is async.
 - A video call is limited in people: You can invite anybody into an async conversation at any time in an issue. You don't have to know who the relevant parties are ahead of time. Everyone can contribute at any time. A video call is limited to invited attendees (and those who have accepted).
 - You can easily “promote” an async conversation from an issue to a video call, as needed. The reverse is harder. So there is lower risk to start with an async conversation.

- For a newcomer to the conversation, it's easier and more efficient to parse an issue, than read a video transcript or watch it.
 - Conversations in issues are easily searchable. Video calls are not.
3. Try to have your video on at all times because it's much more engaging for participants.
- Do not worry if you [can't pay attention at the meeting](#) because you're doing something else, you are the manager of your attention. The flip-side of being the manager of your own attention is that others should not hesitate to request your attention when it is needed.
 - During internal calls, it's okay to eat on video if you're hungry or the call is during your lunch time (please turn your mic off). To maintain professionalism, if you are presenting or facilitating a customer call please try to avoid eating. If eating during a customer call is unavoidable, please turn off your video and mute your mic.
 - You should ensure that you are properly dressed for all video calls. Properly dressed means that you are wearing clothing that covers the top and bottom parts of your body. We do not have a strict dress code policy, but want to make sure that all participants on video calls feel comfortable. If you cannot be properly dressed for the entirety of the call, you should not join, but watch the recording at a later time.
 - Having pets, children, significant others, friends, and family visible during video chats is encouraged. If they are human, ask them to wave at your remote team member to say ["Hi in your native language"](#).
 - Do not feel forced to have your video on, use your best judgement.
4. Additional points for video calls with customers or partners:
- Results come first. Your appearance, location and background is less important than making customers successful so don't wait for the perfect time / place when you can engage a customer right away.
 - Communicating that GitLab is an enterprise grade product and service provider is supported by the way you present yourself. Most of the time, if you would not wear something or present yourself in a certain way at a customer's office, candidate interview, or partner meeting in person then it's probably not the right choice on a video call with them either.
 - Green screens are a great background solution. It's great to work in your garage or basement! Just get a green screen behind you and put up a professional background image to present well externally and still use the rest of the room how you want!
5. We prefer [Zoom](#).
6. Google Calendar also has a [Zoom plugin](#) where you can easily add a Zoom link for a video call to the invite.
7. For meetings that are scheduled with Zoom:

- After reviewing our [Zoom](#) handbook page, if you have additional questions, please contact IT in the `#it_help` Slack channel.
 - Note that if you select to record meetings to the cloud (setting within Zoom), you must include the text `[REC]` in the meeting title if you want them to be automatically placed in the `GitLab Videos Recorded` folder in Google Drive on an hourly basis via a [scheduled pipeline](#).
 - You can find these videos in Google Drive by looking under `Shared drives` and `GitLab Videos Recorded`. If you do not have access to this drive, contact [IT Ops](#).
 - The [script for syncing the files is here](#).
 - Note also that after a meeting ends, Zoom may take some time to process the recording before it is actually available. The sync to Google Drive happens on the hour mark, so if the recording is not available, it may take another hour to be transferred.
8. Consider using a utility to easily mute/unmute yourself, see [Shush](#) in the tools section.
 9. [Hybrid calls are annoying](#).
 10. Always be sure to advise participants to mute their mics if there is unnecessary background noise to ensure the speaker is able to be heard by all attendees.
 11. We start on time and do not wait for people. People are expected to join no later than the scheduled minute of the meeting (before :01 if it is scheduled for :00). The question 'is everyone here' is not needed.
 12. It feels rude in video calls to interrupt people. This is a situation where we have to do something counter intuitive to make all-remote meetings work. In GitLab, everyone is encouraged to interrupt the speaker in a video call to ask a question or offer context. We want everyone to contribute instead of a monologue. Interrupting can be done by clicking "Raise Hand" in Zoom, by physically raising your hand on video, or by interrupting verbally. As a speaker, allow yourself to be interrupted. As a listener, advocate (verbally if necessary) for those who have raised their hand. Just like in-person meetings be cognizant of when, who, and how you interrupt, we don't want [maninterrupting](#).
 13. We end on the scheduled time. It might feel rude to end a meeting, but you're actually allowing all attendees to be on time for their next meeting.
 14. Do not use the chat of products like Zoom to communicate during the call, use the linked document instead. Zoom Chat has been configured for a 30 day retention. This allows everyone to contribute additional questions, answers, and links in the relevant place. It also makes it easier for people in conflicting timezones to contribute questions before the call and makes it easier to review questions and answers after the call, which can be before watching the recording.
 15. You do not need to remind other people to vocalize their questions. Just say their name and a keyword of the question, e.g. 'Jay about credit-cards'.

16. Every comment is document worthy, even small support comments such as +1 or Very Cool! .
17. We encourage the recording and [sharing of everything to our YouTube Unfiltered channel](#).
18. It is unusual to smoke in an open office or video conference, vaping is associated with this. For this reason we ask that you don't vape during calls, and if you absolutely have to, kindly switch your camera off.
19. Speak up when you notice something is not working. If you notice someone's microphone, web cam or latency is causing issues for them it is good to speak up. On a video call it can be harder for the speaker to notice that they aren't being understood compared to a face to face conversation. As such you will be doing them a favour by speaking up early to let them know that they are having a problem. Also see [Hear nothing say something](#) for further explanation.

You are the manager of your attention

You are the manager of your attention, and you decide when you do or don't pay attention in a meeting.

You will always have more work than time in your life. If you get invited to a meeting you don't think you should go to, you should decline the meeting. It is better to cancel than to show up and not pay attention.

On the other hand, not every part of a meeting is relevant, but it can sometimes be helpful to have more people in a call. If you only have one discussion point, if possible, try to reorder the meeting agenda to have your point first and then drop from the call. If you get asked a question when you're not paying attention, it is an okay use of time to repeat a question every now and then. If training is required for one's role, team members should plan to give the training full attention—especially if engagement in discussions or breakout rooms is required. If training is 'nice to learn' or 'optional' for team members, multi-tasking can be done at the team members discretion.

We don't use the first 15 minutes of a meeting to read the materials like they [do at Amazon](#). You can use the start of a meeting to review the materials for the meeting if you need to, given you do not have to be paying attention, but that should not delay the start of the meeting for the people that already have questions based on the materials. [Meetings start on time at GitLab](#).

Do not use your camera to signal you're not paying attention; [cameras should always be on](#).

Do not ask meeting attendees to pay attention

There are too few hours in a week, so we expect each team member to [manage their attention](#). If you're hosting a meeting, don't tell people to give you their attention or stop multi-tasking. Respect each team member's agency over their time. Instead of demanding attention, earn participants' attention by organizing and facilitating meetings so they are compelling to attendees.

First post is a badge of honor

You should take pride in being the first person to add a question to a meeting agenda, however unlike [the First post](#) meme we do want the first post to be more than just "First!". The meeting DRI will be happy to see there is a question ready before to kick off the meeting. The Meeting DRI should remember to thank the person for asking the first question.

Do not do a countdown before ending a call

Never do a countdown or say something like. "I'll give it x seconds", people are very unlikely to ask a question if you do that. Either ask for a question, wait for a question, or end the call.

Hybrid calls are annoying

In calls that have remote participants everyone should use their own equipment (camera, headset, screen).

When multiple people share equipment the following **problems arise for remote participants**:

- Can't hear the sharing people well
- Background noise since the microphone of the sharing people is on all the time
- Can't clearly see facial expressions since each face takes up only a small part of the screen
- Can't easily see who is talking since the screen shows multiple people
- Hard getting a word in since their delay is longer than for the sharing people

The **people sharing equipment also have problems** because they don't have their own equipment:

- Can't easily screen share something themselves
- Trouble seeing details in screen sharing since the screen is further away from them
- Can't scroll through a slide deck at their own pace

- Sharing people can't easily participate (view or type) in a shared document with the agenda and meeting notes.

The disadvantages for remote people are much greater than for the sharing people and hard to notice for the sharing people. The disadvantages cause previously remote participants to travel to the meeting to be in person for a better experience. The extra travel is inefficient since it is time consuming, expensive, bad for the environment, and unhealthy.

Theoretically you can have multiple people in a room with their own equipment but in practice it is much better to be in separate rooms:

- It is annoying to first hear someone talk in the room and then hear it over audio with a delay.
- It is hard to consistently mute yourself when not talking to prevent someone else's voice coming through your microphone as well.

Types of meetings

Ask Me Anything meetings

[Ask Me Anything meetings](#) can be a useful opportunity for team members to meet a new leader, learn more about an existing team member, or gain clarity on a recent change.

1. Format: AMAs use the whole meeting time for questions from attendees, answered by the host.

Fireside Chats

Fireside chats are informal conversations between a host and a guest. The guest is typically a new leader, board member, or guest speaker. They are a useful opportunity to learn specific information about these individuals and their professional careers and personal interests. Fireside chats allow the audience to learn more about the guests in a casual and approachable setting.

Format: In advance of the call, the host will prepare questions and share them with the guest. During the call, the host will moderate the conversation with the guest, by verbalizing the prepared questions. There is specific amount of time reserved at the end of the agenda for questions from attendees.

Walk and Talk calls

A Walk and Talk call is when team members step away from their computers and get outside for a meeting. The difference between a [coffee chat](#) and a Walk and Talk call is that a Walk and Talk call be held with people that you interact with frequently at GitLab. It could be social in nature or focused on a specific problem/topic that needs to be discussed. If it's a problem-solving focused discussion, the outcome should be captured in a merge request. It should not be used if the problem being discussed requires screen sharing or detailed note taking. There are great physical and mental health benefits to a walk and talk call. There are also benefits with [increased focus and creativity](#). A Walk and Talk can also help prevent [Zoom fatigue](#).

The team members can use Zoom on their mobile device with the audio only function, or call one another from their preferred mobile device. A walk and talk call should be agreed to in advance to ensure that the local weather is compatible for a walk in both locations and that the walk and talk call fits into both team members' schedules. We've created a Slack channel [#walk-and-talk-meetings](#) where, if you'd like, you can share pictures from your walking meetings.

Release retrospectives and kickoffs

After GitLab releases a new version [every month](#), we have a 30-minute call a few days later reflecting on what could have been better:

1. What went well this month?
2. What went wrong this month?
3. What could we have done better?

We spend the first part of the retrospective meeting reviewing the action items from the previous month.

On the 8th of each month (or the next business day), we have a kickoff meeting for the version that will be released in the following month. The product team and other leads will have already had discussions on what should be prioritized for that release. The purpose of this kickoff is to get everyone on the same page and to invite comments.

Both the retrospectives and kickoffs are [live streamed to our GitLab Unfiltered YouTube channel](#) and posted to our [Unfiltered YouTube channel](#).

Deep Dives

As GitLab continues to grow, sharing knowledge across the community becomes even more important. The [Deep Dives page](#) describes initiatives we are trying to encourage. This aligns with how we work since everything at GitLab is [public by default](#).

Daily Sync Escalation Process

GitLab has a specific process to follow in crisis situations to ensure effective communications. Details can be found in the [internal handbook](#).

Presentations

1. All presentations are made in Google Slides using [our templates](#).
2. Please allow anyone at GitLab to edit the presentation (preferred) or at least comment on the presentation.
3. If the content can be public use File > Publish to the web > Publish to get a URL and paste that in the speaker notes of the first slide (commonly the title slide).
4. The title of every slide should be the message you want the audience to take away, not the subject matter. So use 'Our revenue more than doubled' instead of 'Revenue growth'.
5. Slide titles should not be more than one line. Be concise in highlighting the key point that you want to make.
6. Do not add a period at the end of a title.
7. During introductions, make sure that nobody is presenting. We remember people better and have more empathy when we clearly see peoples faces and expressions.
8. At the end of the presentation, when you go to Q&A, stop presenting in Zoom. This way the other people can see the person who is speaking much better.
9. All presentations at GitLab should be based on screenshots of the handbook, issues, merge requests, review apps, and data from GitLab Insights and Sisense charts. In most cases it shouldn't be needed to make content uniquely for the presentation. If you need something that doesn't exist yet add it to the place it belongs and then copy it into the presentation. This way we can have a [Single Source of Truth](#) for everything we do. By using screenshots you indicate to people you did the right thing and they can find the canonical source in the appropriate place. Having to find information by digging through old presentations doesn't scale. Consider linking the screenshot to the original source.
10. Do not use cumulative graphs internally. For example total ARR, total user, total contributors, or total Merge Requests. Instead use IACV per dollar spend, users added per month, contributions per month, or [MR rate](#). Cumulative graphs can [hide trends](#) and are [far more likely to be misinterpreted](#). The only acceptable use of cumulative graphs is for external presentations where they are expected by the audience and commonly used.
11. When your presentation includes graphs or other data, make sure your graphs have clear titles and dimensions. Make sure your data has significant figures and labels. For example, use 'January: 1.95M projects' instead of 'January: 1.95M'. Keep

in mind that your audience may not have the full context, especially if they are reading the presentation asynchronously.

12. When giving a presentation, your commentary should not be a regurgitation of the words in the slide. The audience can read the slide for themselves; your commentary should focus on the most important takeaways.

Video and presentation tips with Lorraine Lee

On 2022-01-20, the L&D team hosted [Lorraine Lee](#) for a live speaker series on video and presentation techniques in an all-remote workspace.

Key points addressed in the training include:

- **Increase your confidence on video** using lighting, visuals, and curating your environment
- **Keep your audience engaged** with movement, power words, and the think/do/feel matrix
- **Stay connected with your audience** by smiling, making eye contact, and framing yourself in video
- **Insert energy** by standing up, projecting, and using hand gestures

Watch the replay below:

Live Speaker Series - Lorraine Lee, All-Remote Presentation a
GitLab Unfiltered



Watch on

Writing style guidelines

As you're creating external or business content for GitLab, please refer to the [GitLab Content Style Guide](#). For technical content, you can reference [this word list](#).

This list offers additional guidance for written communication at GitLab:

1. Do not use rich text, it makes it hard to copy/paste. Use [Markdown](#) to format text that is stored in a Git repository. In Google Docs, use "Normal text" using the style/heading/formatting dropdown and paste without formatting.
2. Read our [Markdown Style Guide](#) for more information when using Markdown.
3. Do not use ALL CAPS because it [feels like shouting](#). However, there is the [#all-caps Slack channel](#) for your good-natured shouting needs.
4. We use Unix style (`\n`) line endings, not Windows style (`\r\n`), please ensure `*.md text eol=lf` is set in the repository's `.gitattributes` and run `git config --global core.autocrlf input` on your client.
5. When specifying measurements, please include both Metric and Imperial equivalents.
6. When mentioning currency amounts that team members may need to convert to their local currency (e.g. benefits, expenses, or bonuses), link those amounts to our [Exchange Rates](#) section (e.g. [500 USD](#)).
7. Monetary amounts shouldn't have one digit, so prefer \$19.90 to \$19.9.
8. GitLab is an internationally diverse company. US-based team members should not refer to team members outside the US as "international". Instead, use "non-US". Please also avoid the use of "offshore/overseas" to refer to non-American continents.
9. If you have multiple points in a comment or email, please number them. Numbered lists are easier to reference during a discussion over bulleted lists.
10. When you reference an issue, merge request, comment, commit, page, doc, etc. and you have the URL available, please include it.
11. In making URLs, always prefer hyphens to underscores, and always use lowercase.
12. The community includes users, contributors, core team members, customers, people working for GitLab Inc., and friends of GitLab. If you want to refer to "people not working for GitLab Inc.", just say that, and don't use the word "community". If you want to refer to people working for GitLab Inc., you can also use "the GitLab Inc. team", but don't use "GitLab Inc. employees".
13. When we refer to the GitLab community (excluding GitLab team members), please say "wider community" instead of "community".
14. All people working for GitLab (the company) are the [GitLab team](#). We also have the [Core team](#) that consists of volunteers.
15. Please always refer to GitLab Inc. people as GitLab team members, not employees.
16. Use [inclusive and gender-neutral language](#) in all writing.

17. Always write “GitLab” with “G” and “L” capitalized, even when writing “GitLab.com”, except within URLs. When “gitlab.com” is part of a URL, it should be lowercase.
18. Do not use a hyphen when writing the term “open source” except where doing so eliminates ambiguity or clumsiness.
19. If an email needs a response, write the answer at the top of it.
20. Use the future version of words, just like we don’t write internet with a capital letter anymore. We write frontend and webhook without a hyphen or space.
21. Our homepage is <https://about.gitlab.com/> (with the about. and with https).
22. If you use headers, properly format them (## in Markdown, “Heading 2” in Google Docs); start at the second header level because header level 1 is for titles. Do not end headers with a colon. Do not use emoji in headers as these cause links to have strange characters.
23. Do not use acronyms when you can avoid them. Acronyms have the effect of excluding people from the conversation if they are not familiar with a particular term. Example: instead of MR , write merge request (MR) .
24. If acronyms are used, expand them at least once in the conversation or document and define them in the document using [Kramdown abbreviation syntax](#). Alternatively, link to the definition.
25. If you don’t know the meaning of an acronym, ask. It’s always ok to [speak up](#).

Communicating dates and time

1. For external content, refer to the guidance in the [GitLab Content Style Guide](#).
2. For internal content, use ISO dates: yyyy-mm-dd
3. For months, use yyyy-mm , so 2018-01 for January.
4. Refer to a year with CY18 (never with 2018) and a quarter with CY18-Q1 to prevent confusion with fiscal years and quarters. If the year is obvious from the context it is OK to use Dec 4, but not 12/4.
5. GitLab operates on a [Fiscal Year](#) offset from the calendar year. When referring to a fiscal year or quarter, please use the following abbreviations:
 1. “FY20” is the preferred format and means: Fiscal Year 2020, the period running from February 1, 2019 through January 31, 2020
 2. “Q1” = the first quarter of the current Fiscal Year, so on Feb 1, 2020, “Q1” is the period from Feb. 1, 2020 through April 30, 2020. Note that Epics in GitLab follow Calendar Years and Quarters.
 3. When referring to a quarter in a future or past year, combine the two above: “FY21-Q1”
 4. When financial data is presented, include a note to indicate fiscal year (e.g. “Fiscal Year ending January, 31 ‘YYYY’”)
6. For time formatting guidelines, refer to the [GitLab Content Style Guide](#).

7. Please remember that not everyone is working in the same timezone; what may be morning for you is evening for someone else. Try to say 3 hours ago or 4 hours from now, or use a timestamp, including a timezone reference.
8. Don't use "EOD" or "end of day" ("COB" or "close of business", "EOB" or "end of business"), unless you are okay with a deliverable being due at the end of [anywhere on earth](#). Team members communicate across timezones, where "end of day" does not specify the exact date and time. When you want something due at a specific time, [communicate the date and time](#) by when the request should be done, for example: Please review before 2023-06-10 5PM PT .

Visuals

Many times an explanation can be aided by a visual. Whenever presenting a diagram, we should still allow everyone to contribute. Where possible, take advantage of the handbook's support for [Mermaid](#). If you are new to using Mermaid or need help troubleshooting errors in your Mermaid code, the [Mermaid Live Editor](#) can be a helpful tool. Where taking advantage of Mermaid isn't possible, link to the original in our Google Drive so that the diagram can be edited by anyone.

Ubiquitous language

At GitLab we use [ubiquitous language](#) to increase communication efficiency.

This is defined in [Domain-driven design](#) as: "A language structured around the domain model and used by all team members to connect all the activities of the team with the software."

We use it for activities in GitLab, even ones not implemented in software.

By having ubiquitous words to identify concepts we prevent confusion over what is meant, for example we refer to [parts of our organization](#) as a function, department, or group depending on exactly what is meant.

Make sure that domains don't overlap, for example [organization size](#) and [deal size](#) don't reuse words to prevent overlap.

If a term is ambiguous don't use it, for example our hiring team uses the terms `roles` and `vacancies` , but avoid the ambiguous word `job` .

Make sure that projects and working groups have clear and direct names. Prefer "CI Spend Reduction Working Group" to "Project Raven Working Group".

Make sure that people can infer as much as possible from the word, for example our [subscription options](#) allow you to know if someone is using self-managed or GitLab.com.

Make sure terms don't overlap without clearly defining how and why, for example see our [tier definitions](#).

Keep terms to one or at most two words to prevent people from introducing ambiguity by shortening a term. When using two words make the first word unique because people tend to drop the second word more often.

MECEFU terms

MECEFU is an acronym for Mutually Exclusive Collectively Exhaustive Few words Ubiquitous-language.

You pronounce it: MessiFu. Think of the great soccer player Lionel Messi and his [kung fu](#) or soccer fu skills.

We want to use MECEFU terms to describe a domain to ensure efficient communication. MECEFU terms have 4 characteristics that help with efficiency:

1. Mutually Exclusive: nothing is referenced by more than one term
2. Collectively Exhaustive: everything is covered by one of the terms
3. Few words: the longer terms are the more likely it is people will not use all of them and cause confusion, therefore consider two words as the upper limit for a single term. Avoid acronyms because they are hard to remember (we're open to a few words to replace MECEFU as an acronym :)
4. Ubiquitous language: [defined above](#)

An example of a MECEFU term is our [sales segmentation](#):

1. Mutually Exclusive: There is no overlap between the numbers and there is a single dimension.
2. Collectively Exhaustive: Everything for 0 to infinite employees is covered.
3. Few words: Mid-market is a natural combination and SMB is abbreviated.
4. Ubiquitous language: We're not using the word 'Enterprise' which already can refer to our Enterprise Edition distribution.

One nit-pick is that the Medium of SMB and Mid of Mid-Market sound very similar.

Simple language

Simple Language is meant to encourage everyone at GitLab to simplify the language we use. We should always use the most clear, straightforward, and meaningful words possible in every conversation. Avoid using "fluff" words, jargon, or "corporate-speak" phrases that don't add value.

When you **don't** use Simple Language, you:

- Confuse people and create a barrier for participants of your conversation.
- Cause others to not speak up in a meeting because they don't understand what you're saying.
- Are not inclusive of those whose first language is not English.
- Do not add value with your words.

When you **do** use Simple Language, you:

- Get work done more efficiently.
- Build credibility with your audience (your team, coworker, customer, etc.).
- Keep people's attention while you're speaking.
- Come across more confident and knowledgeable.

Here's an example:

Original sentence

We're now launching an optimization of our approach leveraging key learnings from the project's postmortem.

A Simple Language sentence

We're creating a new plan based on what we learned from this project.

Simple Language is important both when we're speaking to other team members and when we're representing GitLab to people outside the company.

Be sure to use Simple Language in written communications as well. Our handbook, website, docs, marketing materials, and candidate or customer emails should be clear, concise, and effective. Corporate marketing maintains guidelines on [GitLab's tone of voice](#).

Instead of...	Try...
Getting buy-in/Getting alignment	Asking for feedback since DRIs make decisions
Synergy	Effective Collaboration
Get all your ducks in a row	Be organized
Do not let the grass grow too long	Work quickly
Leverage	Use more explicit phrasing- debt, etc.
Send it over the wall	Share it with a customer

Instead of...	Try...
Boil the ocean	Waste time
Punt	Make less of a priority
Helicopter view/100 foot view	A broad view of the business
Turtles all the way down	Cascade through the organization
When someone has spare/extra cycles	When someone is available

Inefficient things shouldn't sound positive

For example, do not suggest that you're "working in real-time" when a matter is in disarray. Convey that a lack of organization is hampering a result, and provide feedback and clear steps on resolving.

Do not use a cool term such as "tiger team" when the [existing term of "working group"](#) is more exact. While cool terms such as these may be useful for persuading colleagues to join you in working towards a solution, the right way isn't to use flowery language.

The last example is when we used 'Prioritizing for Global Optimization' for what we renamed to headcount reset. When we [renamed it](#) we saw a good reduction in the use of this disruptive practice of moving people around.

Using additional languages

Using American English as our standard language supports our values such as efficiency, results, and transparency. Careful use of another person's language can be a celebration of diversity and build an atmosphere of inclusion.

The guidance in this section applies to written one-to-one communication, for example, merge request comments between an author and reviewer, not merge request descriptions or commit messages. Also keep the following in mind:

- Use of an additional language is optional.
- Stick to the few simple phrases in the table below.
- Always include the language used and a translation to English.
- Team members can choose to indicate the languages they speak in their Slack profile.
- When in doubt, use American English.

Here's an example:

Hey @nmalcolm, I left some suggestions for your merge request. Ka mau te wehi!
(Te Reo Māori: great work / well done!)

ありがとうございます (Japanese: thank you very much) for the review @cynthia!

Language	Hello	Thank you	Great work / well done
Croatian		Hvala	
Japanese		ありがとうございます	
Te Reo Māori	Kia ora	Ngā mihi	Ka mau te wehi!

Avoid using Git in Project Names

Avoid using *Git* in the naming of internal and external company related programs (BagGit, GitFit, Gitty, GitIt, etc.). Referencing Git creates an inaccurate perception that GitLab has a narrow focus. While GitLab started as a source control platform, it has become [The DevOps Platform](#).

Email

We have a low internal email culture, as we see greater efficiency in other forms of communication (e.g. Slack). If you are emailing, please use the following guidelines:

1. Send one email per subject as multiple items in one email will cause delays (have to respond to everything) or misses (forgot one of the items).
2. Always reply to emails by replying to all, even when no action is needed. This lets the other person know that you received it. A thread is done when there is a single word reply, such as OK, thanks, or done.
3. If you're sending an email to a large group of people (or a distribution list), put those recipients in BCC (rather than in the "To" field) so that a reply all won't ping hundreds of people.
4. If you forward an email without other comments please add FYI (for your information), FYA (for your action), or FYC (for your consideration). If you forward an external request with FYC it just means the person who forwarded it will not follow up on the request and expects you to decide if you should follow up or not, the terms comes from [movie promotion to voters](#).
5. Email forwarding rules are specified in the shared [Email, Slack, and GitLab Groups and Aliases](#) Google Doc accessible only to people in the company. If you want to be added or removed from an internal alias, change a rule, or add a forwarding email alias, please [suggest an edit](#) in the doc and [submit a new access request](#).

6. Only Google Workspace domain admins are allowed to provision Google Groups and email distributions.
7. Emails are asynchronous, for example, if your manager emails you on a weekend it is fine to reply during the workweek.
8. If an email is or has become **urgent** feel free to ping people via chat referencing the subject of the email.
9. If you or your team needs to send an email to a group of team members, not grouped in a current [Google email group](#), and specifically related to personally identifiable information (location, state, country, etc.) please contact a Total Rewards Analyst at total-rewards@gitlab who can create an email list from Workday data, with approval.
10. Where appropriate, consider using professional salutations including Hi or Hello and avoid colloquial expressions such as Hey, Oh, or Sup. Sometimes only the person's name is suitable. The level of formality should often mirror the formality from previous messages when communicating with internal team members as well as external persons.
11. Try to always use a person's name when starting or responding to a message, especially if there are multiple persons cc'd, so that the addressee knows you are addressing them.
12. Make sure all relevant letters and words that need capitalization are capitalized, such as the start of sentences or the word "I".
13. Proofread your messages so that sentences are punctuated correctly, typos are fixed, and grammar is corrected. Consider using the really helpful [Grammarly](#) tool - this tool is great for both native English speakers and for those who use English as an additional language.
14. All messages and replies are signed with a professional send-off (ex. Best regards), your name, and your signature block.

Slack

Slack is used for:

- Internal-only communication and announcements impacting all team members
 - Linking a GitLab Issue or Merge Request (confidential if necessary) where discussion can take place is encouraged when sharing announcements on Slack
 - Consider creating a [Manager Mention MR](#), so that we can collaborate with Efficiency and Transparency across threads
- Company newsletters and team updates
- Asynchronous team standups, questions, and quick collaboration

- Informal communication

Use a [bias for action](#) to quickly move conversations that require collaboration and action out of Slack and into an issue.

Only 90 days of Slack activity will be retained, so Slack should specifically NOT be used for:

- Obtaining approvals
- Documenting decisions
- Storing official company records or documents
- Sharing personal or sensitive information regarding any individuals

Internal Slack messages between team members are still considered professional communication. Please do not use or add emoji's to Slack that are of a political, religious or of a sexual nature. You can refer to the [Religion and politics at work](#) section of the handbook. When in doubt do not use or add the emoji. If you have any concerns about an emoji that was used, please reach out to the author or if you are not comfortable doing so please reach out to your [People Business Partner](#).

There is a lot of information pertaining to Slack, as it is a critical part of GitLab's communication. See the [Slack tools and tips page](#).

General guidelines

1. Everyone can contribute, and while opinions are important to provide perspective, we value [proposals](#) and [iteration](#). If the subject is of value to the wider community, consider commenting on an existing issue or opening a new [merge request](#) instead.
2. Use the `:white_check_mark:` emoji or similar to indicate an inquiry has been answered. Anyone can add the emoji. If you're not sure, then feel free to leave it up to the person who asked. An emoji indicator is particularly helpful in channels where lots of questions are posted, such as `#questions`, and `#git-help`.
3. In general, you can think of emoji reactions as equivalent to body-language responses we use in real-life conversations, such as nodding your head as encouragement when a verbal (or in Slack, written) response might be too much. However, please be aware that use and understanding of emoji, like body-language, is not universal. Others may not communicate via those means the same way that you do, particularly those who are Autistic or otherwise neurodivergent. If in doubt, you can use text to clarify.
4. In both public and private channels, [threads](#) are valuable for keeping conversations together. If you want to respond to a question or comment in a channel, please start a thread instead of responding below them in the channel. This helps to keep the discussion in one place where it is easy to follow, and reduces noise as each

- message in a thread does not result in an unread message for everyone in the channel.
5. Unless you're in an active chat, don't break up a topic into multiple messages as each one will result in a notification which can be disruptive. Use [threads](#) if you want to provide extra info to the question/comment you posted.
 6. If you are having a hard time keeping up with messages, you can update your preferences to have Slack email you all notifications. To change the setting, go to `Preferences > Notifications > When I'm not active on desktop...` and "send me email notifications."
 7. If you agree in a message to start a video call (typically by asking "Call?") the person that didn't leave the last comment starts the call. So either respond to the "Call?" request with a video link or say "Yes" and let the other person start it. Do not say "Yes" and start a call 5 seconds later since it is likely you'll both be creating a video call link at the same time.
 8. As an admin of the Slack workspace, if given the option to "*Disable future attachments from this website*" when removing an attachment from a message **this will block the link/domain from [unfurling](#) in the entire Slack workspace**. Be careful and deliberate when choosing this option as it will impact every user in the workspace.
 9. When referencing a Slack thread in a GitLab.com issue, don't *only* link to the thread. Not only will people outside of the GitLab organization be unable to access the content, but the link will expire after the Slack retention period expires. Instead:
 1. **Review the contents** for confidentiality of users, customers, or any other sensitive information before posting.
 2. Copy and paste the relevant parts of the thread into the issue using blockquote formatting.
 3. Link to the Slack thread and include `(internal)` after the link. For example: <https://gitlab.slack.com/archives/C0AR2KW4B/p1555347101079800> (internal)
 4. Post a link to the issue note in the Slack thread to let others know that discussion has moved to the issue.
 10. When selecting your Slack display name, please do not have your name in all capital letters as this is often [associated as shouting](#) in written communications.

Avoid direct messages

Note: We don't use the term *private message*, because these *direct messages* are not inherently *private* like a phone call or private letter. The messages are potentially accessible by Workspace admins or via Backups. Slack refers to these types of messages as [direct messages themselves](#).

When using Slack for work-related purposes, please avoid direct messages. [Direct messages discourage collaboration](#). You might actually be contacting the wrong person, and they cannot easily redirect you to the right person. If the person is unavailable at the moment, it is less efficient because other people cannot jump in and help. Use a public channel and mention the person or group you want to reach. This ensures it is easy for other people to chime in, involve other people if needed, and learn from whatever is discussed.

If someone sends you a work-related direct message, it is okay to let them know you'd like to take the conversation to a public channel, linking to this section of the handbook. The process might look something like:

- In the direct message: Thanks for reaching out, that's a great question/idea I think the rest of the team could benefit from. I'm going to move this to #public-channel based on [our desire to avoid direct messages] (</handbook/communication/#avoid-direct-messages>)
- In the appropriate public channel: @Person asked "question" in a DM, pulling that out here if anyone else has input.
- Answer the question in a thread on that channel message, allowing others to benefit.

If you find yourself getting a lot of direct messages that should go in a public channel, consider changing your Slack status to an attention grabbing emoji and set it to something like:

- Please consider posting in a public channel before direct messaging
- Why direct message me when you can post in a public channel?

If you must send a work-related direct message, don't start a conversation with "Hi" or "Hey" as that interrupts their work without communicating anything. If you have a quick question, just ask the question directly, and the person will respond asynchronously. If you truly need to have a synchronous communication, then start by asking for that explicitly, while mentioning the subject. e.g., "I'm having trouble understanding issue #x, can we talk about it quickly?".

Do not use group direct messages

Use private channels instead of group direct messages. Group direct messages are very hard to maintain, track, and respond to. They also have a key limitation in that you can't add people to the conversation. This is a hindrance to collaboration and transparency.

Consider whether the conversation can take place in a public channel. If not, please use a private channel instead. This channel may have a short-term purpose. It is

acceptable to leave the channel and/or archive it if you are no longer an active participant or the channel is no longer in use.

As we grow headcount, we exponentially increase the lines of communication- 3 people have 3 communication lines, 4 have 6, and 41 have 820. As a result, there is a natural tendency for people to prefer private channels of communication. The intentions are good, as people are looking to reduce noise for others, but this can lead to the same problems as described elsewhere on this page, notably:

- Communication is siloed.
- As we grow, people may be reaching out to the wrong person.
- If you have a question, other people might have it too.

Slack is our primary source of chat communication and is where many personal interactions happen. We want to continue to encourage folks to build personal relationships with one another which will often happen over DMs.

Use public channels

- If you use Slack and plan to message 3 or more people, we recommend a channel for customer/issue/project/problem/partnership.
- Learn about [common channels and channel-naming conventions](#).
- If something is important but not urgent - like complimenting or encouraging the entire team - use email or post in the channel without @-mentioning the team.
- It's not rude to leave a channel. When you've had your questions answered or are no longer interested, feel free to leave the channel so it won't distract you anymore.
- The usage of ChatBots for integrations can sometimes depend upon the name of the channel. You should consult the channel about such integrations before changing the name of commonly used/popular channels to avoid inadvertently breaking integrations.

Be respectful of others' time

Start by understanding what we mean by respecting time. We should err toward putting material into channels over DMs and public channels over private channels even though we understand that this will generate more messages that can be read by more people. Respecting time is not about reducing the overall volume of channel messages that team members receive. It's about making sure that messages are targeted, expectations for asynchronous responses are clear, and we are communicating with consideration.

The following tips provide ways to work respectfully with others given this context, though is not an exhaustive list:

- If you're only referring to someone, but don't actually need their attention, and want to spare them from getting notified, spell out their name normally without @ mentioning them.
- You also do not need to @ mention someone if they are part of a Slack thread unless you need their attention (for them to review, respond, etc.), since Slack has a dedicated view for threads.
- Slack messages should be considered asynchronous communication, and you should not expect an instantaneous response; you have no idea what the other person is doing.
- Do not feel obligated to respond to Slack messages when you are not working.
- Feel free to send a colleague a link to these guidelines if the communication in Slack should be done **asynchronously**.
- **Please avoid using @here or @channel unless this is about something urgent and important.** In chat, try to keep the use of keywords that mention the whole channel to a minimum. They should only be used for pings that are both urgent and important, not just important. By overusing channel mentions, you make it harder to respond to personal mentions promptly since people get pinged too frequently. Additionally, if you are planning to @mention a specific team ([Slack User Group](#)), consider the size of the group you are mentioning ([see group membership](#)) and the impact of pinging all of these people for the particular situation. If something is urgent and important:
 - Use @here to notify all currently *active* members in the room. Please only use @here if the message is important *and* urgent.
 - Use @channel to notify *ALL* members in the room, irrespective of away status. Please only use @channel if the message is important *and* urgent.
- If you are aware that your teammate is on vacation, avoid mentioning them in a high volume channel. It will be difficult to find the information or question when they return. If you need to ensure they refer back to the thread, ensure to send them a link to the relevant Slack message through a direct message.

Questions

If you have a question that you can't find the answer to in our handbook (or you need help finding something in the handbook) team members across the company are here to help. Go directly to the subject matter experts/source in the [designated slack channel](#) to ensure your question is addressed.

If your question doesn't relate to any of the existing topics:

- Ask it in the #questions Slack channel

- Once you receive an answer, document it in the handbook and post the MR link in your question thread
-  once you've been helped

When would GitLab use Corporate Export?

The times this feature would be used would be to comply with certain obligations. Corporate Export must be enabled by Slack in accordance with Slack's policy, which can be found [here](#).

Examples of instances where GitLab may need to use this feature may include, but are not limited to, those situations listed in [Slack's documentation](#).

Are my direct messages and private channel conversations completely private?

No. The Slack Workspace Owner has the ability to export data from all direct messages and private channel conversations for the maximum retention period set by GitLab, which is currently set for 90-days. All messages that are older than 90-days cannot be exported by the Workspace Owner or any other Team Member at GitLab. While messages are not actively monitored, GitLab reserves the right to monitor its software for the reasons stated in its [Employee Privacy Policy](#), including, but not limited to, the safety and protection of our Team Members, the protection of our intellectual property, and the exercise or defense of legal claims.

Please keep [GitLab values](#) in mind when communicating directly with other team members. If you have a confidential personal issue that you do not feel comfortable discussing via a business-provided internal communications tool, it is recommended to use a personal form of communication such as a text message or phone call. For additional questions, please address in the [issue](#).

Emergency chat

Slack is down

To use the "Slack Down!" group chat on Zoom:

1. In the Zoom desktop app go to the `Team Chat` tab
2. Click `+`
3. Click `Join a Channel`
4. Search `Slack down!`
5. Click `Join`

Once service is restored, go back to Slack.

Zoom is down

To use Slack Huddle:

1. Navigate to the appropriate Slack channel or direct message.
2. Use `/huddle` to trigger a huddle, which supports both audio and video calls.
3. You may need to give permissions if it's the first time you are using Slack calls.

Once service is restored, go back to Zoom.

Slack and Zoom are down

Join the [Slack Down!](#) room on Hangouts Chat. Once service is restored, go back to Slack and Zoom.

Types of documents

Google Docs

Never use a Google Doc / Presentations for something non-confidential that has to end up on the website or the **handbook**. Work on these edits via commits to a merge request. Then link to the merge request or diff to present the change to people. This prevents a duplication of effort and/or an out of date handbook.

Google Docs can be useful when rapidly [iterating](#)/commenting/suggesting on the content, but if the content is meant to be long lived it should be moved to the handbook as an SSOT and deprecated with a link to the handbook page. If the content is short lived, e.g. one-time report that won't be referred to beyond 2-3 weeks, it can remain in a Google Doc or [presentation](#).

Pageless is the GitLab preferred format

Google Docs [Pageless format](#) is the preferred format for company documents that won't be printed. If you set your [default to Pageless](#) then this will be applied to all future documents as well. If a document is likely going to be printed (for example, a contract) the older paged style is acceptable. See [Good practices and helpful tips](#) for help navigating the pageless format.

Link sharing

If you *do* need a Google Doc, create one with your company Google Workspace (formerly G Suite) account and set the visibility, access controls and searchable flag according to the following guidelines.

The recommended defaults when sharing a document for GitLab internal purposes is setting visibility to **GitLab**, access to **Editor** and searchable **Can find in search results** to ensure everyone can contribute!

Note: To our knowledge, it is not possible to set the default to **Can Edit** and you have to change the permissions from **View** manually. We hope that Google adds this capability in the future.

Visibility Setting	Use Cases
Anyone with the link	Avoid this setting. Instead, choose GitLab , then explicitly share the document with desired external individuals. Only use this if you want the document to be public but not indexed by Google.
GitLab (Recommended Default)	This is the recommended default as it allows anyone within GitLab to easily discover documents via searching for their name within Drive.
Restricted	When the document contains highly sensitive or private information such as 1:1s with direct reports
Searchable Setting	Effect
_____	_____
Can find in search results.	makes the file available to anyone with a PeppGitLaberdine Google account and discoverable within GitLab Google Drive's search feature.
Must have link to access	makes the file available to anyone with a GitLab Google account who has the link.
Access Setting	Use Cases
_____	_____
Editor	Anyone that can view the document can edit it. This is the recommended setting when On - GitLab is enabled for the document
Commenter	Anyone that can view the document can add a comment but cannot edit the document. This is ideal if you want to provide

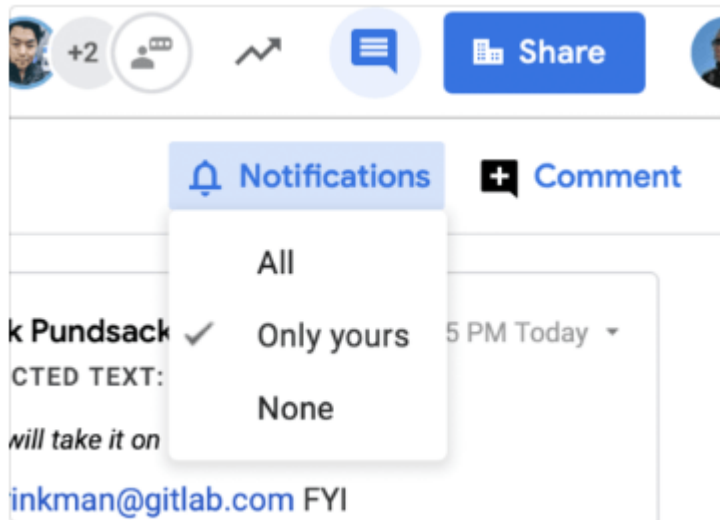
Visibility Setting	Use Cases
	visibility but retain more fine-grained control of document editing.
Viewer	Individuals with access to the document will only be able to view it.

Reference Google's [documentation](#) on Link Sharing to learn more.

Good practices & helpful tips

1. If you have content in a Google Doc that is later moved to the website or handbook, [deprecate the Google Doc](#).
2. When referring to a Google Doc or folder on the Google Drive in the handbook, refrain from directly linking it. Instead, indicate the name of the doc. If you link the URL people from outside the organization can request access, creating workload and the potential for mistakes. (In the past, linking a Google Doc has led to inadvertently opening the sharing settings beyond what was intended.) This also helps prevent spam from people outside GitLab requesting access to a doc when clicking its link. To save people time you can also [link to the search results page](#) which allows people to quickly get to the doc without anyone being able to request access. If there are multiple documents showing up in the search, you may filter your [search link by adding the owner](#).
3. If you are having trouble finding a shared Google Doc, make sure you [Search <your domain>](#) in Google Drive.
4. In our handbook, if you find yourself wondering whether it is better to provide a public link to a Google Doc vs. writing out the content on the website, use the following guideline: Is this document frequently adapted / customized? If yes, then provide a link, making sure that the document can be *commented on* by *anyone* with the link. For instance, this is how we share our employment [contracts](#). If the document is rarely customized, then provide the content directly on the site and deprecate the Google Doc.
5. If you want to quickly find where a team member's cursor is in a Google Doc, click their icon at the top of the document and the app will jump you to the current location. This works in Sheets and Presentations as well.
6. You can set the notifications on a Google Doc so you only get emailed when someone tags you directly instead of getting emails for every comment. Click on "notifications" and select "Only yours". By the way, when you create the doc, it defaults to All, but when you are just shared with it, it defaults to Only yours. There

is [no global default](#).



7. You can find a template of the [GitLab letterhead](#) on the shared Google Drive. Please be sure to make a copy of the document to your Google Drive before customizing the template for your use.
8. If you want to have the Google Doc be readable to the public, do not change the sharing settings from 'Editable by Everyone at GitLab', [publish the document](#) instead.
9. In all cases, the sharing settings (who a document is shared with, if it is visible to the whole company, etc.) on a Google Doc should be the single source of truth for the confidentiality of the document.
10. Avoid using your initials when adding content to Google Docs or referring to someone else. Use your full name, as Google Docs [Smart Compose](#) will auto-complete names for you, and display information about the GitLab team member on hover. It makes documents more readable and consumable, especially for people outside your team/department.
11. When there is a synchronous discussion about an issue or MR captured in a Google Doc, be sure to summarize the discussion into the issue or MR and link to the agenda item where it was discussed with a [bookmark](#). If the notes are [limited access](#) or no notes were taken, please say so. This will ensure that interested GitLab Team Members can see both the summary and context of the discussion.
12. In the [pageless format](#) you can use COMMAND+OPTION+SHIFT+J or **View→Hide Comments** to quickly hide comments that reduce document width.
13. When using Google Docs, paste full URLs rather than linking text. [Chips](#) are the one acceptable alternative when linking to another Google document. Advantages for using URLs rather than hyperlinks include:
 1. Greater visibility when it is a long URL and specific. Links are important but easy to miss otherwise (short and only a different color and/or underlined).
 2. Ability to see what the content is without hovering over (which isn't quick on mobile).

3. Makes it easier to paste the content as formatting free text (not rich text), which we prefer and is sometimes the only option.

How to deprecate a Google Doc

1. Add 'Deprecated: ' to the start of the title.
2. Remove the content you moved.
3. Add a link to the new location at the beginning of the doc/first slide/first tab.
4. Add a link to the merge request or commit that moved it (if applicable).

Handbook page

Pages like this are part of the [handbook](#). The GitLab handbook is the central repository for how we run the company.

Product documentation

[GitLab Docs](#) - docs.gitlab.com is where you can find documentation on GitLab, the product.

GitLab repositories

`repository` / `repositories` are where files are kept under [source code management](#). In most cases, requires MRs to merge. The Handbooks are in a repository, our code is in a repository, etc.

READMEs

README - README.md files are what are shown by default when you browse a repository. Contains useful information to give context on what the project/repository are for. These can also be used for [user profiles](#) as personal READMEs.

Google Calendar

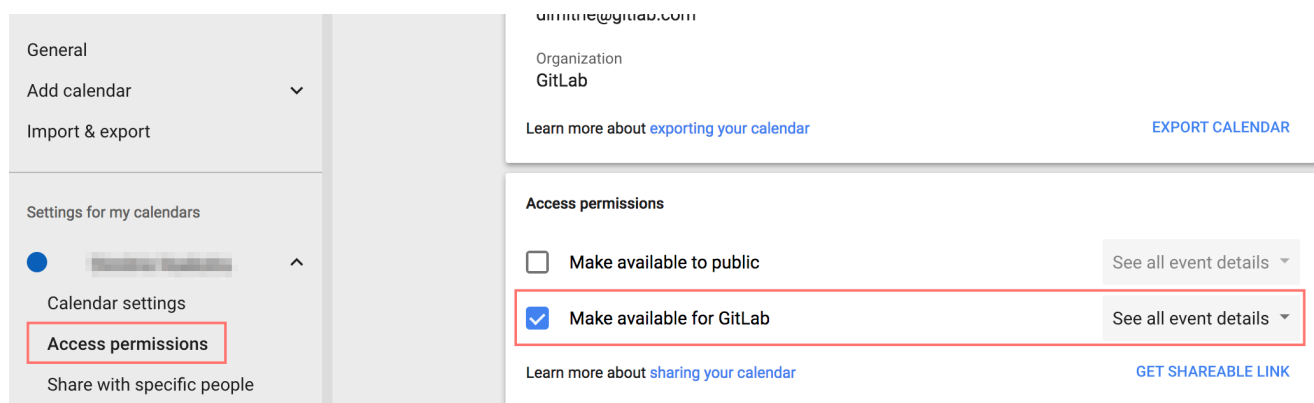
We recommend you set your Google Calendar access permissions to 'Make available for GitLab - See all event details'. Consider marking the following appointments as 'Private':

- Personal appointments
- Confidential & sensitive meetings with third-parties outside of GitLab
- 1-1 performance or evaluation meetings

- Meetings on organizational changes

There are several benefits and reasons to sharing your calendar with everyone at GitLab:

- Transparency is one of our values and sharing what you work on is in line with our message of “be open about as many things as possible”.
- Due to our timezone differences, there are small windows of time when our availabilities overlap. If other members need to schedule a new meeting, seeing the details of recurring meetings (such as 1-1s) will allow for more flexibility in scheduling without needing to wait for a confirmation from the team member. This speaks to our value to be more efficient.



If you add blocks of time spent on recurring tasks to your Google Calendar to remind yourself to do things (e.g. “Check Google Analytics”), consider marking yourself “Free” for those events so that coworkers know they may schedule a meeting during that time if they can’t find another convenient time.

Google Calendar Appointment Scheduling

This feature allows you to create a link to an availability schedule that you can send to your customers or coworkers for them to schedule a call according to your availability.

This allows you to only show available spots while keeping your other calls private. This also avoids having to go back and forth between you and other person figuring out what day and time works best for both of you. Since this is a native Google Calendar functionality, there is no need to set up integrations with your calendar like other scheduling tools.

A member of our Customer Success team created a [demo video of how to use this feature](#).

External communication

Key practices to consider during any meeting are listed below.

1. Video Calls - If this is your first time meeting a customer/prospect/partner/etc., turn on your camera when you login to Zoom. This will help to make the customer/prospect feel more comfortable as they are certain your undivided attention is geared towards them.
2. “No agenda, no attenda” - Always have an [agenda prepped and ready to go](#), with the exception of [coffee chats](#). Share this with your audience. Make sure that everything on the agenda is accurate and ask if there’s anything missing that needs to be addressed during this call or for the future. When there is no agenda, it translates to you not caring. When sharing agendas with customers and partners it should be called “GitLab + Name Shared Collaboration & Agenda”, not “External Agenda” as that implies there is something the parties are not seeing.
3. 70/30 Rule - Ask open ended questions that leave the audience talking 70% of the time, while you are talking 30% of the time. Please note that this varies based on the type of meeting that you are conducting. Be conscious of what questions need to be asked and to capture those items.
4. Take Notes - Effective note-taking is a valuable skill that will help you retain and recall any important details. Be the person who remembers all the details of your audience’s needs.
5. Adapt to Audience Tone - Before going into the business portion of your meeting, evaluate first the tone of the audience. Adapt your tone accordingly in order to appeal to various types of personalities.
6. Mid-call - Half-way through the meeting, check in with your audience. Ask them what their thoughts are on the progression of this meeting and if what you’re presenting is on the right track. This helps both you and the audience by re-aligning expectations and making sure the meeting is going the right direction.
7. Pre-Close Summary - 10 Minutes (1-hour meetings) or 5 minutes (30 minute meetings) prior to ending the call, ask the audience to build out an agenda for the next step or meeting. This helps to secure next steps and to ensure there are no balls dropped.
8. Post Meeting Action - Immediately write down notes and next steps and input into the proper directory (Google Drive, Salesforce, etc.).
9. Two Block Rule - For in person meetings with external parties you should wait until you’re more than two blocks from the meeting before discussing the results of the meeting. Nobody wants to hear themselves being discussed in the bathroom.

Communicating with media and industry analysts

GitLab team members are not authorized to speak with the media or analysts on behalf of our company unless authorized by our Marketing department. Unless authorized, do not give the impression that you are speaking on behalf of GitLab in any communication that may become public. This includes posts to online forums, social media sites, blogs, chat rooms, and bulletin boards. This policy also applies to

comments to journalists about specific matters that relate to our businesses, as well as letters to the editor and endorsements of products or services.

For more, please visit the [Corporate Communications handbook section](#).

Paid external speaking requests

GitLab as the leader in all remote work creates opportunities for our team members to receive requests from external 3rd parties to participate on panels, blogs or news publication or articles. Recently our team members have been approached by external 3rd parties looking to pay or compensate GitLab team members for their time to discuss GitLab remote practice to help them guide a client. Other third parties may contact GitLab team members to provide subject matter expertise that they may have by virtue of their role at GitLab. As in any request we ask that team members verify who they are speaking with to make sure the source is indeed a valid and legitimate request. Always remember that you represent GitLab and if any question makes you uncomfortable or gives you a pause on whether you should answer then we recommend that you do not answer. A third party's questions regarding GitLab financials, sales, compliance, executives or specifically where the company is heading should be treated with the most caution. We want and encourage all team members to be remote evangelists and this can be done without giving very specific information about GitLab. If you have any concern about a request please reach out on slack to [#external-comms](#)

Social media

Please see our [team member social media policy](#).

Posting or streaming to YouTube

See the [YouTube page](#) for options and instructions for posting recordings and live streaming to our YouTube channels.

Guidelines for vendor meetings

1. We request external vendor meetings to use our video conferencing tool so we can quickly join the call and record the session if we need to. Confirm with vendor that they agree we can record the call. The DRI for the vendor meeting will generate the zoom link and share with the vendor.
2. Decide ahead of the meeting who should be invited, i.e. those likely to get the most out of it.
3. Ahead of the meeting, we should agree internal agenda items/requirements/priorities and provide to the external provider.

4. In order to make the best use of time, we wish to avoid team introductions on the call, particularly where there are a number of us attending. We can include a list of attendees with the agenda and give it to the vendor before or after the meeting. Introductions can be helpful in some external calls. In those meetings, use [these guidelines](#).
5. When a question or issue is raised on the agenda, if the person who raised it is present they will verbalize it live on the call; or if they are not present, then someone will raise it for them. This is a common GitLab practice.
6. Where possible, we request that the vendor provides their slides / presentation materials and any other related information after the meeting.
7. Do not demo your tool live, create a pre-recorded walk-through of the tool and make it available to GitLab before the meeting so we can ask questions if needed.
8. Be cognizant of using inclusive language.
9. We respectfully request that everyone is mindful of the time available, to help manage the call objectives effectively.

User communication guidelines

- Keep conversations positive, friendly, real, and productive while adding value.
- If you make a mistake, admit it. Be upfront and be quick with your correction. If you're posting to a blog, you may choose to modify an earlier post. Just make it clear that you have done so.
- There can be a fine line between healthy debate and incendiary reaction. Try to frame what you write to invite differing points of view without inflaming others. You don't need to respond to every criticism or barb. Be careful and considerate.
- [Assume positive intent](#) and explicitly state the strongest plausible interpretation of what someone says before you respond, not a weaker one that's easier to criticize. [Rapoport's Rules](#) also implores you to list points of agreement and mention anything you learned.
- Answer questions, thank people even if it's just a few words. Make it a two way conversation.
- Appreciate suggestions and feedback.
- Do not make promises that you can't keep.
- Do not offer [platitudes](#). Be direct and respectful. For [example](#) - we don't need to suggest that our actions are because we "heard feedback" when they are really done for other reasons.
- Guide users who ask for help or give a suggestion and share links. [Improving Open Development for Everyone](#), [Types of requests](#).
- When facing negative comments, respond patiently and treat every user as an individual, people with the strongest opinions can turn into [the strongest supporters](#).

- By default, discussions in issues and MRs are public and could include participation of wider community members. It is important to make the wider community members feel welcome participating in discussions and sharing their view. Wider community members also submit [MRs to help improve our website/handbook](#) and this is often their first contribution to GitLab. We want to make sure that we are responsive to their contributions and thank them for helping improve GitLab.
- Adhere to the [Code of Conduct](#) in all communication. Similarly, expect users to adhere to the same code when communicating with the GitLab team and the rest of the GitLab community. No one should accept being mistreated.

Company phone number

If you need to provide the details of GitLab's contact information you can take the [address from the visiting page](#) for reference; or the [mailing address](#) of the office in the Netherlands if that is more applicable.

If a phone number is required, leave this field empty by default. If that is not possible, then use the general number (+1-650-667-4852), but be aware that this number simply guides to a voice message that refers the caller back to contacting us via email.

Effective communication competency

[Competencies](#) are the Single Source of Truth (SSoT) framework for things we need team members to learn.

In an all-remote organization effective communication is key to exchanging knowledge, ideas, and information. Effective communication at GitLab:

- Uses [asynchronous](#) communication as the starting point and stays as open and transparent as we can by [communicating via text](#) through public issues, merge requests, and Slack channels (over DMs).
- Places an emphasis on ensuring that conclusions of offline conversations are written down ensuring a [single source of truth](#).
- [Produces video](#) when necessary.

If you would like to improve your skills or expand your knowledge on topics relating to Communication at GitLab, check out our resources:

- [Communicating effectively and responsibly through text](#)
- [Embracing asynchronous communication](#)

Skills and behavior of applying effective communication as a Team Member:

- Effectively practices communication via text.
- Uses asynchronous communication when possible: [merge requests](#) (preferred) or [issues](#).
- Directs all communication to the appropriate channels (Slack, GitLab, email).
- Recognises when synchronous communication is the more appropriate option.
- Directs all decisions and discussions to the Handbook as a single source of truth.
- Records videos to communicate information when that is the most efficient and effective way to consume the content.
- Employs multimodal communication to broadcast important decisions.
- Practices low context communication and provides as much background as possible when communicating via text to avoid confusion.

Skills and behavior of applying effective communication as a People Manager:

- Implements working asynchronously across teams, departments or across the company. Drives communication where possible to asynchronous channels.
- Holds team members accountable for effectively communicating via text.
- Fosters an environment across teams, departments or divisions where asynchronous communication is the starting point.
- Guides team members on when producing video is appropriate. Implements interactive communication tools across their team, department or the company depending on level.
- Drives and funnels conversations to the right channels across teams, divisions and the company.

[Ask Me Anything](#)

Learn and ask questions at GitLab's Ask Me Anything (AMA) meetings

[Confidentiality levels](#)

At GitLab, we are public by default, but some information is classified as internal or limited access. This page provides details on confidentiality levels.

[Deep Dives](#)

The goal of a Deep Dive session is to share knowledge about a particular topic, with the assumption that recipients already have some basic understanding or aptitude for the given subject.

[GitLab Communication — Zoom](#)

Zoom-specific communication guides GitLab uses Zoom as the primary video collaboration platform for ...

[GitLab Communication Chat](#)

Introduction At GitLab, Slack is critical to our communication with each other. While it enables ...

[Power of the Pause](#)

Power of the Pause There are times in everyone's life when you can feel overwhelmed, anxious, ...

[Top Misused Terms - GitLab Communication](#)

List of Top Misused Terms Below are terms people frequently use when they should use another term. ...

Last modified May 12, 2026: [Update file _index.md to add async v sync resource \(aeeb959b\)](#).

 [View page source](#) -  [Edit this page](#) - please [contribute](#). 